

Extractie van gestructureerde data uit PDF-bestanden met large language models met een beperkt aantal parameters: een vergelijking van tekstgebaseerde, multimodale en hybride benaderingen.

Kyandro Voet.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Mevr. V. Depestele

Co-promotor: Dhr. E. Duka

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Dit is de bachelorproef waarmee ik mijn opleiding Toegepaste Informatica aan HOGENT afsluit. Het schrijven van deze thesis was voor mij een leerrijke en uitdagende ervaring, niet alleen op technisch vlak, maar ook wat betreft het zelfstandig opzetten en uitvoeren van een onderzoek.

Het onderwerp van deze bachelorproef ontstond uit een probleem dat mijn stagebedrijf Turtle SRL had. Er werd dagelijks een groot aantal facturen verwerkt via een geautomatiseerde pipeline die gebruik maakt van Google Gemini. Hoewel deze aanpak goed werkt, zijn er toch duidelijke beperkingen: de bedrijfsdata moet doorgestuurd worden naar externe servers, de verwerkingstijd loopt op bij grote volumes en de kosten stijgen mee met het aantal verwerkte documenten. Die observatie bracht mij op de centrale vraag van dit onderzoek: kan een lokaal taalmodel, dat volledig door Turtle SRL gehost wordt, een evenwaardig alternatief zijn hiervoor?

Wat mij in dit onderzoek aansprak, was een combinatie van de volgende problemen: de nood aan het efficiënt verwerken van facturen en werken met Large Language Models, iets wat momenteel snel ontwikkelt. Dit is voor mij dan ook een mooie kans om mijn kennis hiervan te verdiepen en hiermee aan de slag te gaan. Wat het ook interessant maakt is dat dit onderzoek gericht is op het uitwerken van een oplossing op consumenten hardware, wat betekent dat het eigenlijk door elk bedrijf zou kunnen gebruikt worden.

Dit onderzoek had ik niet kunnen uitvoeren zonder de hulp van verschillende mensen. Eerst en vooral wil ik mijn promotor, Mevr. V. Depestele, en mijn co-promotor, Dhr. E. Duka, oprecht bedanken voor hun begeleiding, hun constructieve feedback en de vrijheid die zij mij gaven om dit onderwerp grondig te verkennen. Daarnaast wil ik ook mijn stagebedrijf, Turtle SRL bedanken voor het bieden van de mogelijkheid om dit onderzoek uit te voeren en voor hun ondersteuning gedurende het hele proces.

Samenvatting

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Inhoudsopgave

Lijst van figuren	viii
Lijst van tabellen	ix
Lijst van codefragmenten	x
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.2.1 Deelvragen met betrekking tot het probleemdomein:	2
1.2.2 Deelvragen met betrekking tot het oplossingsdomein:	2
1.3 Onderzoeksdoelstelling	2
1.4 Opzet van deze bachelorproef	3
2 Stand van zaken	4
2.1 Huidige werkwijze Turtle SRL	4
2.2 Waarom PDF-documenten lastig zijn	5
2.3 Problemen met de huidige cloud-aanpak.	5
2.4 De verschuiving naar lokale modellen	6
2.5 Ollama als hosting platform	6
2.6 Drie benaderingen voor documentverwerking	7
2.6.1 1. De multimodale aanpak (Vision-Language Models)	7
2.6.2 2. De tekst-gebaseerde aanpak (OCR + Tekst-LLM)	8
2.6.3 3. De hybride aanpak (Lay-out Analyse)	8
2.7 Selectie van het tekstmodel	8
2.8 Selectie van het vision model	9
2.9 Het model finetunen.	10
2.9.1 1. Volledige Fine-Tuning	10
2.9.2 2. Parameter-Efficiënt Fine-Tuning	11
2.9.3 De evolutie naar QLoRA voor lokale hardware.	11
2.9.4 Waarom er voor QLoRA gekozen wordt	12
2.10 Evaluatiemethodes.	12
2.10.1 Nauwkeurigheidsmetrieken	13
2.10.2 Operationele prestatie-metrieken	14

3	Methodologie	15
3.1	Literatuurstudie	15
3.2	Vorbereidingsfase	16
3.3	Implementatiefase	16
3.4	Evaluatie- en Conclusiefase	17
4	Vorbereidingsfase	18
4.1	Opzet van de testomgeving	18
4.2	Dataset en ground truth	19
4.2.1	Samenstelling van de dataset	19
4.2.2	Testdataset	19
4.2.3	Trainingsdataset	20
4.2.4	Gecombineerd overzicht	20
4.2.5	Variatie en representativiteit	21
4.2.6	Categoriespecifieke JSON-schema's	21
4.2.7	Aanmaak van de ground truth	22
4.2.8	Evaluatiemethode	23
4.2.9	Representativiteit	23
4.3	Selectie van de modellen	23
4.3.1	Tekstmodel: Qwen3-8B	23
4.3.2	Visionmodel: Qwen3-VL:8B	24
4.3.3	Motivatie voor de gemeenschappelijke modelfamilie	24
5	Uitvoeringsfase	25
5.1	Fine-tuning van de modellen	25
5.1.1	Overzicht en motivatie	25
5.1.2	Vorbereiding van de trainingsset	25
5.1.3	Fine-tuning van het tekstmodel	27
5.1.4	Fine-tuning van het visionmodel	28
5.1.5	Beperkingen	28
5.2	De tekstgebaseerde aanpak	29
5.2.1	Overzicht	29
5.2.2	Stap 1: Tekstextractie	29
5.2.3	Stap 2: Extractie van informatie met het taalmodel	29
5.2.4	Stap 3: Verwerking van de output	30
5.2.5	Stap 4: Opslag	30
5.3	De multimodale aanpak	31
5.3.1	Overzicht	31
5.3.2	Stap 1: PDF naar afbeeldingen	31
5.3.3	Stap 2: Aanbieding aan het vision model	32
5.3.4	Stap 3: Verwerking van de output en opslag	33

5.4	De hybride pipeline.	33
5.4.1	Overzicht	33
5.4.2	Stap 1: Lay-outanalyse.	34
5.4.3	Stap 2: Selectieve PDF naar afbeelding conversie	34
5.4.4	Stap 3: Tesseract blokanalyse voor gescande pagina's	35
5.4.5	Stap 4: Visuele zones naar het vision model	35
5.4.6	Stap 5: Tekstcontext samenstellen	36
5.4.7	Stap 6: Definitieve JSON-generatie.	36
5.4.8	Stap 7: Verwerking van de output en opslag.	37
5.4.9	Beperkingen.	37
6	Conclusie	38
A	Onderzoeksvoorstel	40
A.1	Inleiding	40
A.2	Literatuurstudie	41
A.3	Methodologie	43
A.3.1	Literatuurstudie.	43
A.3.2	Vorbereidingsfase	44
A.3.3	Implementatiefase.	44
A.3.4	Evaluatie- en Conclusiefase	45
A.4	Verwacht resultaat, conclusie	45
	Bibliografie	47

Lijst van figuren

Lijst van tabellen

4.1	Verdeling van de testdataset per categorie	19
4.2	Verdeling van de trainingsdataset per categorie (gecorrigeerd)	20
4.3	Gecombineerd overzicht van test- en trainingsdataset	20
5.1	QLoRA-parameters voor het tekstmodel	27
5.2	Trainingsparameters voor het tekstmodel	27

Lijst van codefragmenten

1

Inleiding

In bijna alle bedrijven zijn PDF-documenten, zoals facturen, nog steeds een cruciale bron van administratieve informatie (Smallpdf, 2024). Deze documenten bevatten vaak essentiële gegevens zoals bedragen, factuurnummers, btw-tarieven en klantgegevens, die belangrijk zijn voor de boekhouding en rapportage. Ondanks de digitalisering van veel bedrijfsprocessen, blijft het automatisch verwerken van PDF-bestanden inefficiënt en foutgevoelig. Dit komt onder andere door de verschillende opmaakstijlen en lay-outs die deze documenten vaak hebben.

1.1. Probleemstelling

Dit onderzoek focust zich op dit probleem vanuit het perspectief van Turtle SRL, een bedrijf dat gespecialiseerd is in het optimaliseren van bedrijfsprocessen.

In de huidige situatie maakt Turtle SRL gebruik van een geautomatiseerde pipeline waarbij documenten via OCR-technieken zoals Tesseract worden verwerkt in combinatie met Google Gemini 2.5 Flash. Hoewel deze aanpak werkt, zijn hier ook een aantal problemen mee.

Ten eerste is er een probleem op het gebied van privacy, omdat bedrijfsgevoelige data naar Google verstuurd moet worden. Daarnaast zorgen de kosten per token en de verwerkingstijd voor schaalbaarheidsproblemen. Het duurt momenteel 10 tot 20 seconden om een factuur te verwerken.

De doelgroep van dit onderzoek is in de eerste plaats Turtle SRL aangezien het hun probleem oplost. Daarnaast kunnen de resultaten van dit onderzoek ook interessant zijn voor IT-professionals en data engineers die verantwoordelijk zijn voor het implementeren en automatiseren van het verwerken van documenten binnen hun organisaties.

1.2. Onderzoeksvraag

Om de hierboven beschreven problemen op te lossen, staat de volgende onderzoeksvraag centraal in dit onderzoek: "Wat is de meest geschikte oplossing voor het accuraat en efficiënt extraheren van data uit facturen met behulp van lokale LLM's wanneer een tekstgebaseerde, multimodale en hybride aanpak vergeleken wordt?"

Om een volledig antwoord te kunnen geven op deze hoofdvraag, is het onderzoek opgesplitst in specifieke deelvragen.

1.2.1. Deelvragen met betrekking tot het probleemdomein:

- Hoe is de huidige werking met Google Gemini en waar liggen de specifieke limieten?
- Welke impact heeft het gebruik van Google Gemini op de privacy van het bedrijf?
- Hoe kan de variëteit in invoerformaten de foutgevoeligheid van de huidige werking beïnvloeden?
- Welke soorten PDF-bestanden bestaan er en hoe verschillen deze in opmaak en structuur?

1.2.2. Deelvragen met betrekking tot het oplossingsdomein:

- Wat is het verschil in accuratesse tussen de tekstgebaseerde, multimodale en hybride aanpak?
- Hoe efficiënt zijn de methoden in termen van verwerkingstijd en geheugen-gebruik, vergeleken met de huidige cloudoplossing?
- In welke mate is elke benadering robuust bij het verwerken van facturen met onbekende layouts en formaten?
- Wat is de optimale balans tussen modelgrootte en prestaties?
- Welke fine-tuning techniek is het meest geschikt voor LLM's met een beperkt aantal parameters in dit specifieke domein?

1.3. Onderzoeksdoelstelling

Het hoofddoel van deze bachelorproef is het onderzoeken, lokaal implementeren en objectief evalueren van de drie benoemde benaderingen aan de hand van een realistische dataset van facturen.

Het concrete eindresultaat van dit onderzoek bestaat uit twee delen:

- Een werkende proof-of-concept (PoC) die aantoonst hoe de drie verschillende architecturen werken.
- Een adviserend onderzoeksrapport met een vergelijkende studie.

Het onderzoek zal succesvol zijn wanneer er een duidelijke aanbeveling kan worden gedaan aan Turtle SRL over de meest geschikte aanpak.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4 worden de voorbereidingen voor de implementatie beschreven, waaronder de opzet van de testomgeving en de selectie van de modellen.

In Hoofdstuk 5 worden de implementaties en de uitgevoerde experimenten gedetailleerd beschreven.

In Hoofdstuk 6 worden de resultaten van de experimenten geanalyseerd, waarbij de tekstgebaseerde, multimodale en hybride oplossingen met elkaar vergeleken worden op basis van accuraatheid en snelheid.

In Hoofdstuk 7, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

In deze literatuurstudie wordt de huidige stand van zaken rond de automatische verwerking van ongestructureerde documenten geanalyseerd. Ten eerste wordt er nagegaan hoe Turtle SRL op dit moment deze documenten verwerkt, hierbij wordt er ook gekeken naar de technologiekeuzen en de problemen die hierbij komen kijken. Vervolgens worden er drie verschillende benaderingen besproken die gebruikt kunnen worden om deze problemen aan te pakken met behulp van large language models (LLM's). Hierbij zal er ook onderzocht worden welke fine-tuning technieken er bestaan voor LLM's met een beperkt aantal parameters (enkele miljarden). Daarnaast zal er bekeken worden welke soorten PDF-bestanden er bestaan en hoe deze verschillen in termen van opmaak en structuur.

2.1. Huidige werkwijze Turtle SRL

Bij Turtle SRL komen verschillende soorten documenten binnen, zoals PNG, PDF, XML, DOCX, TXT en XLSX. Het doel is om de informatie uit deze documenten te halen en deze te ordenen in een gestructureerd JSON-formaat, daarna worden deze opgeslagen in een MongoDB databank. Dit proces verloopt in een aantal vaste stappen. De eerste stap is om elk document, ongeacht het type, om te zetten naar een gewone tekst.

- Digitale documenten (zoals XML, DOCX en digitale PDF's), deze documenten bevatten tekst die direct uitgelezen kan worden.
- Afbeeldingen en gescande documenten (zoals PNG en gescande PDF's), deze documenten hebben een extra stap nodig, Optical Character Recognition (OCR). Turtle SRL gebruikt hiervoor Tesseract, een bekende tool die letters in afbeeldingen kan herkennen en omzetten naar tekst.

Zodra deze tekst klaar is, krijgt het document een label, dit bepaalt welke prompt

naar de LLM gestuurd zal worden. Momenteel wordt hiervoor Gemini 2.5 Flash gebruikt via een cloud-verbinding. Dit model leest de tekst, zoekt de juiste informatie op basis van de prompt, en zet deze in de gewenste JSON-structuur.

2.2. Waarom PDF-documenten lastig zijn

Veel van de binnenkomende documenten, zoals facturen, zijn PDF-bestanden (Smallpdf, 2024). Volgens Saout e.a. (2024) is het extraheren van gegevens uit PDF's erg moeilijk. Een webpagina is logisch opgebouwd met titels en paragrafen, maar een PDF werkt helemaal anders. Deze is eigenlijk ontworpen om er op elk scherm en elke printer exact hetzelfde uit te zien. Een PDF-bestand onthoudt dus alleen waar een letter of een lijn precies op de pagina moet staan, maar begrijpt niet dat de letters een woord vormen of de lijnen een tabel vormen (Stephens, 2022).

Wanneer je een PDF omzet naar platte tekst, gaat al de opmaak verloren (Stephens, 2022). Dit is een groot probleem bij documenten die tabellen of meerdere kolommen bevatten (Saout e.a., 2024). De LLM kan niet weten hoe deze tabel in elkaar zit met enkel de tekst, hierdoor kunnen gegevens verkeerd overgenomen worden.

2.3. Problemen met de huidige cloud-aanpak

Hoewel het gebruik van een cloud-model zoals Gemini 2.5 Flash in het algemeen werkt, botst Turtle SRL in de praktijk op een aantal grote problemen:

- Privacy en beveiliging: Gevoelige bedrijfsgegevens, zoals facturen, moeten nu naar de server van Google gestuurd worden. Dit kan leiden tot privacyrisico's en kan in strijd zijn met de wetgeving van de GDPR, omdat het bedrijf de data niet meer volledig zelf in de hand heeft.
- Snelheid: Nu duurt het gemiddeld 10 tot 20 seconden om een document te verwerken. Als er veel documenten tegelijk verwerkt moeten worden, kan dit leiden tot vertragingen.
- Kosten en afhankelijkheden: Bij cloud-modellen betaal je per verwerkte factuur. Hoe meer documenten het bedrijf verwerkt, hoe hoger deze kosten zullen zijn. Bovendien ben je volledig afhankelijk van de prijzen van de externe provider. Als Google morgen beslist om de prijs te verdubbelen, kan je hier niets aan doen.
- Kwaliteit en gevoelige prompts: Algemene modellen zoals Gemini zijn getraind voor allerlei verschillende taken, hierdoor kan het zijn dat deze informatie missen die nodig is voor ingewikkelde bedrijfsdocumenten. Om toch goede resultaten te krijgen, moeten de prompts heel precies geschreven worden. Dit is erg kwetsbaar: een kleine aanpassing in de prompt kan er voor zorgen dat het model fouten maakt of de JSON-structuur kapotmaakt. Dit

toont aan dat er behoefte is aan een model dat speciaal getraind is voor deze specifieke taak.

2.4. De verschuiving naar lokale modellen

Om de nadelen van cloud-gebaseerde modellen (zoals de trage verwerkingstijd en de privacyrisico's) op te lossen, zal er gekeken worden naar lokale LLM's. Dit betekent dat het model op de eigen servers van het bedrijf zal draaien. Hierdoor is er geen internetverbinding met een externe partij nodig, wat betekent dat de data volledig binnen het bedrijf blijft. Bovendien heb je geen variabele kosten meer, er wordt eenmaal een server aangekocht waarop de LLM wordt gedraaid, en daarna is alles gratis. Omdat de grootste modellen te veel rekenkracht vereisen om lokaal te draaien, is er een sterke trend richting modellen met een beperkter aantal parameters, vaak enkele miljarden. In dit onderzoek zal er gekeken worden hoe goed kleinere modellen zijn met behulp van finetuning.

2.5. Ollama als hosting platform

Om lokale taalmodellen eenvoudig te kunnen draaien op consumenten hardware bestaan er verschillende platforms die het hosten en uitvoeren van LLM's vereenvoudigen (SitePoint, 2026). Een veelgebruikt voorbeeld hiervan is Ollama, een open-source platform dat het lokaal uitvoeren van taalmodellen sterk vereenvoudigt (SitePoint, 2026). Ollama heeft een interface waarmee modellen met één commando gedownload en uitgevoerd kunnen worden, dit kan ook zonder internetverbinding of externe API (IntraGPT, 2023).

Intern maakt Ollama gebruik van llama.cpp, een C++-bibliotheek die geoptimaliseerd is voor efficiënte op zowel CPU als GPU (Mastery, 2025). Een belangrijk kenmerk van Ollama is de ondersteuning voor het GGUF-formaat (GPT-Generated Unified Format), een bestandsformaat dat gekwantiseerde modellen opslaat in een compacte representatie (Hugging Face, 2025). Dankzij kwantisatie kunnen modellen met meerdere miljarden parameters worden teruggebracht tot een grootte die past binnen het beperkte VRAM-budget van consumenten hardwaren (Mastery, 2025). Ollama ondersteunt verschillende kwantisatieniveaus, waarbij q4_k_m een veelgebruikte keuze is die een goede balans biedt tussen modelkwaliteit en geheugengebruik (Hugging Face, 2025). Bij dit niveau worden de modellen opgeslagen als 4-bit getallen in plaats van de standaard 16-bit of 32-bit (Hugging Face, 2025). Dit zorgt ervoor dat het geheugengebruik drie tot vier keer minder is ten opzichte van het originele model (Master, 2026).

Naast de ondersteuning voor tekstgebaseerde modellen biedt Ollama ook ondersteuning voor multimodale modellen die zowel tekst als afbeeldingen als invoer kunnen verwerken (Ollama, 2026d). Afbeeldingen worden hierbij aangeboden als base64 gecodeerde strings (Ollama, 2026d). Dit maakt Ollama bruikbaar als inter-

face voor de drie aanpakken.

Ollama heeft een REST API en een Python-bibliotheek waarmee modellen eenvoudig kunnen worden aangesproken vanuit Python-scripts (Ollama, 2026b). De bibliotheek ondersteunt parameters zoals `temperature`, `num_ctx` en `format` om het gedrag van het model aan te kunnen passen (Ollama, 2026a). Voor Qwen3-modellen biedt Ollama ook de `think`-parameter waarmee de `thinking`-module van het model uitgeschakeld kan worden, wat de verwerkingstijd verkort bij taken waarbij dit niet vereist is (Ollama, 2026c).

Alternatieven voor Ollama zijn onder andere vLLM en LM-studio, deze zijn beiden gefocust op het lokaal draaien van LLM's maar met verschillende doelgroepen en technologische focus (Glukhov, 2025). vLLM is geoptimaliseerd voor hoge snelheid op server-grade hardware met meerdere GPU's, waarbij geoptimaliseerde technieken zoals tensor-parallelism en GPU-memory-tuning wordt gebruikt om de beste doorvoersnelheid te realiseren (vLLM, 2026). Deze installatie vereist een uitgebreide configuratie en tuning, waardoor vLLM vooral geschikt is voor productie-omgevingen en minder voor eenvoudige, eenmalige setup-scenarios op consumentenhardware (vLLM, 2026). LM-Studio biedt een grafische interface maar heeft geen volwaardige Python API die bruikbaar is in een geautomatiseerde pipeline (LM Studio, 2024).

2.6. Drie benaderingen voor documentverwerking

Om de data uit facturen en andere documenten te halen, kunnen we op drie manieren te werk gaan met behulp van LLM's: een multimodale, een tekst-gebaseerde en een hybride aanpak.

2.6.1. 1. De multimodale aanpak (Vision-Language Models)

Bij deze aanpak wordt het document, zoals een PDF, niet eerst omgezet naar tekst. In plaats daarvan wordt de pagina als een afbeelding aan het model gepresenteerd. Hierdoor worden Vision-Language Models (VLM's) gebruikt (C., 2025). Deze modellen kunnen niet alleen de tekst lezen, maar zien ook de opmaak, tabellen en afbeeldingen. VLM's bestaan uit twee delen: een vision encoder en het taalmodel. De vision encoder analyseert een afbeelding of PDF en het taalmodel interpreteert deze dan met de geschreven prompt. Het grote voordeel van deze methode is het behoud van de opmaak en tabellen, het model ziet de factuur zoals een mens het zou zien. Hierdoor begrijpt het direct waar een tabel begint en eindigt, en welke velden bij welke kolommen horen. Er zijn ook een aantal belangrijke nadelen voor een lokale implementatie:

- Hoge vereisten rekenkracht: Het verwerken van afbeeldingen vereist veel grafische rekenkracht (Rajendran e.a., 2025).
- Resolutie: Als een document veel kleine tekst bevat, zal de afbeelding soms in een lagere resolutie aan het model gegeven worden om geheugen te be-

sparen, waardoor de tekst onleesbaar kan worden.

- Trage verwerking: Het model heeft meer tijd nodig om een antwoord te geven dan bij een LLM met alleen tekst (Rajendran e.a., [2025](#)).

2.6.2. 2. De tekst-gebaseerde aanpak (OCR + Tekst-LLM)

Deze methode is vergelijkbaar met degene die momenteel gebruikt wordt. Hierbij dient het model enkel als een tekstverwerker. Het document moet dus eerst volledig worden getransformeerd naar een tekst voordat de LLM hier iets mee kan doen. Voor digitale PDF's kan de tekst vaak via softwarebibliotheken worden uitgelezen. Voor gescande PDF's en afbeeldingen is Optical Character Recognition (OCR) vereist (ABBYY, [2025](#)). Tesseract OCR is hierbij een veelgebruikte, open-source software die gebaseerd is op deep learning en gebruikt wordt in zowel academische als industriële toepassingen (ABBYY, [2025](#)). Volgens (Smith, [2007](#)) behaalt Tesseract hoge herkenningsnauwkeurigheid bij standaard gedrukte teksten en ondersteunt het meerdere talen, waaronder Engels, Nederlands en Italiaans. Hierdoor zal er in dit onderzoek ook gebruik gemaakt worden van Tesseract OCR.

De voordelen van deze aanpak zijn snelheid en efficiëntie: pure tekst neemt amper geheugen in beslag, waardoor deze zelfs met kleine modellen snel verwerkt kan worden. Het grootste nadeel is het verlies van lay-out en de structuur (Stephens, [2022](#)). Wanneer een OCR een factuur met twee kolommen leest, leest deze vaak gewoon van links naar rechts. Hierdoor zal de data in een onlogische volgorde aan het model gegeven worden, wat kan leiden tot een foutieve output.

2.6.3. 3. De hybride aanpak (Lay-out Analyse)

Om de nauwkeurigheid van de multimodale aanpak en de snelheid van de tekst-gebaseerde aanpak te combineren, kunnen we gebruik maken van een methode die deze beide combineert. Hierbij zal er eerst een lay-outanalyse uitgevoerd worden op het document. Pdfplumber zal de verschillende onderdelen detecteren en categoriseren. Vervolgens zal het systeem routing toepassen (Rajendran e.a., [2025](#)). De simpele tekstblokken zullen via OCR omgezet worden naar tekst en behandeld worden door de tekst-gebaseerde aanpak. De complexe categorieën zoals tabellen zullen als aparte afbeeldingen behandeld worden door het multimodale model (Rajendran e.a., [2025](#)).

2.7. Selectie van het tekstmodel

Voor de tekst-gebaseerde en de hybride aanpak is er nood aan een puur tekstmodel dat lokaal kan draaien. Bij de keuze van dit model spelen verschillende factoren een rol. Omdat het model op de eigen hardware van Turtle SRL moet draaien, is het VRAM-gebruik een harde beperking. Daarnaast is het belangrijk dat het model meertalig is, want zo goed als alle facturen zullen in het Italiaans zijn. Tot slot is een

sterke instructie-following capaciteit belangrijk, omdat het model een specifieke JSON-structuur moet genereren.

Op basis van deze criteria zijn er een aantal modellen die in aanmerking komen:

- **Llama 3.x (Meta):** Een veelgebruikte open-source reeks met varianten van 8B tot 70B parameters. Voor dit onderzoek is enkel het 8B model relevant aangezien dit het enige model is dat in het VRAM-budget valt („Ollama VRAM Requirements: Complete Guide to Local LLMs on Consumer GPUs”, [2026](#)).
- **Mistral / Mixtral (Mistral AI):** Mistral 7B is een klein model dat sterk presteert op instructietaken. De Mixtral-variant maakt gebruik van een Mixture of Experts-architectuur waardoor het efficiënter omspringt met rekenkracht (Teki, [2024](#)).
- **Qwen 3 (Alibaba):** Qwen 3 is een reeks open-source modellen van Alibaba die beschikbaar zijn in varianten van 0.5B tot 72B parameters. Een belangrijk voordeel van deze reeks is de ondersteuning voor meerdere talen, waaronder Nederlands, Engels en Italiaans (Qwen, [2025](#)).

De definitieve keuze van het model zal gebeuren in de voorbereidingsfase, en zal samen gebeuren met mijn copromotor.

2.8. Selectie van het vision model

Voor de multimodale aanpak is er nood aan een Vision-Language Model (VLM) dat zowel tekst als afbeelding als input kan verwerken. Naast de algemene criteria zoals VRAM-gebruik en meertaligheid is beeldresolutie ook een belangrijke criterium. Indien het model een afbeelding in een te lage resolutie verwerkt zal kleine tekst op een factuur niet goed gelezen kunnen worden.

- **LlaVa (Large Language and Vision Assistant):** LlaVa is een van de eerste open-source VLM's en combineert een visuele encoder met een tekstmodel zoals Llama. De 7B-variant past met 4-bit quantisatie comfortabel binnen 8GB VRAM. Een belangrijk nadeel is dat LlaVa ontworpen is voor algemene visuele taken en minder sterk presteert op documenten met complexe lay-outs zoals tabellen en meerdere kolommen (Yasuno, [2026](#)).
- **Qwen3-VL (Alibaba):** Qwen3-VL is de meest recente multimodale uitbreiding van de Qwen-reeks en ondersteunt zowel tekst als afbeeldingen als input. Een belangrijk voordeel is dat dit model documenten in hoge resolutie kan verwerken dankzij een dynamisch resolutiemechanisme, waarbij de resolutie automatisch aangepast wordt aan de complexiteit van het document. Daarnaast biedt Qwen3-VL ondersteuning voor meerdere talen, waaronder Engels en Italiaans. De 8B-variant vereist slechts 6.1GB VRAM, waardoor het comfortabel

binnen de 8GB limiet van de hardware van Turtle SRL past („Available Models - ComfyUI-QwenVL”, [2025](#)).

- **InternVL2-8B (Shangai AI Lab):** InternVL2 is een open-source VLM-reeks die specifiek geoptimaliseerd is voor documentbegrip en OCR-taken. De 8B-variant past met 4-bit quantisatie binnen 8GB VRAM en scoort sterk op het extraheeren van gegevens uit documenten met complexe lay-outs zoals tabellen en meerdere kolommen (OpenGVLab, [2023](#)).

De definitieve modelselectie zal ook gebeuren in de voorbereidingsfase.

2.9. Het model finetunen

Wanneer een lokaal model ingezet wordt om gestructureerde data uit een factuur te halen, zal deze snel een fout maken. Zelfs met een heel uitgebreide prompt kan het model afwijken van het verwachte resultaat, extra tekst toevoegen of juist velden vergeten. Dit probleem wordt prompt instability genoemd. (Stewart e.a., [2024](#))

Om het model stabiel en betrouwbaarder te maken voor deze specifieke taak, maken we gebruik van fine-tuning („Fine-tuning Models”, [2025](#)). Fine-tuning is het proces waarbij een model dat al getraind is voor algemene zaken, verder getraind wordt op een specifieke dataset („Fine-tuning Models”, [2025](#)). In plaats van het model telkens via de prompt te vertellen wat het moet doen, passen we de interne kennis van het model aan, zodat het standaard de juiste JSON-structuur genereert. Om dit resultaat te verkrijgen zijn er meerdere mogelijkheden.

2.9.1. 1. Volledige Fine-Tuning

Bij volledige fine-tuning worden werkelijk alle parameters aangepast op basis van een nieuwe dataset.

- **Voordeel:** Dit levert in theorie de beste prestaties, omdat het hele model aangepast is naar de nieuwe taak.
- **Nadeel:** Deze methode is heel zwaar. Zelfs voor een klein model met 8 miljard parameters vereist dit heel veel rekenkracht. Hierdoor zouden er extreem dure videokaarten moeten aangekocht worden en zou er veel tijd in moeten gestoken worden (Palomares Carrascosa, [2025](#)). Bovendien bestaat er een risico op catastrophic forgetting. Hierbij vergeet het model zijn algemene kennis omdat alle originele verbindingen zijn overschreven (Practical DevSecOps, [g.d.](#)). Voor een kostenefficiënte, lokale bedrijfsoplossing is deze methode daarom vaak onhaalbaar.

2.9.2. 2. Parameter-Efficiënt Fine-Tuning

Om de nadelen van volledige fine-tuning op te lossen, is er een nieuwe categorie ontwikkeld: Parameter-Efficient Fine-Tuning (PEFT) (Z. Han e.a., 2024). Deze werkt veel efficiënter, in plaats van het volledige model te trainen naar de nieuwe dataset, voegen ze wat extra parameters toe aan het basismodel (L. Xu e.a., 2024). Dit zorgt ervoor dat het model zijn brede, algemene kennis behoudt, terwijl het toch de nieuwe specifieke taak aanleert (Xia e.a., 2023). Dit vereist uiteraard ook minder resources omdat vaak minder dan 1 procent van de parameters wordt toegevoegd (Z. Xu e.a., 2026). Er bestaan ook verschillende manieren om dit te doen:

- **Prefix Tuning:** Bij een standaard prompt schrijven we instructies in gewone mensentaal, dit worden harde prompts genoemd. Bij prefix tuning worden er onzichtbare, wiskundige woorden aan de prompt toegevoegd, dit worden ook wel soft prompts genoemd (Z. Han e.a., 2024). Tijdens het fine-tunen wordt het model niet aangepast, maar zoekt het naar de perfecte reeks van onzichtbare woorden om de beste JSON-output te genereren. Het grote nadeel hiervan is dat deze soft prompts ruimte in beslag nemen in het context window van het model. Omdat facturen soms lange documenten zijn is het zonde om deze ruimte op te offeren aan soft prompts.
- **Adapters:** Dit is een techniek waarbij er kleine, extra laagjes tussen de bestaande lagen van het neurale netwerk worden gebouwd (Z. Han e.a., 2024). Alleen deze kleine tussenlagen worden getraind op bedrijfsdata. Hierbij wordt er geheugen bespaard tijdens het trainen, maar er zijn ook grote nadelen. Als het model nu een antwoord wil geven, moet het eerst door extra lagen gaan, hierdoor zal de verwerkingstijd van het document trager zijn.
- **Low-Rank Adaptation:** Low-Rank Adaptation (LoRA) pakt het probleem slimmer aan en is momenteel de meest succesvolle PEFT-methode. In een neurale netwerk worden berekeningen uitgevoerd via grote matrices. LoRA voegt geen extra lagen toe die alles vertragen, maar plaatst twee kleine matrices naast de originele lagen (Z. Han e.a., 2024). Tijdens het trainen leert alleen dit kleine deel hoe de facturen verwerkt moeten worden. Na het trainen kan dit deel samengevoegd worden met het originele model. Hierdoor is het model even snel als ervoor, maar heeft het wel extra kennis

2.9.3. De evolutie naar QLoRA voor lokale hardware

Wanneer we een model van enkele miljarden parameters lokaal willen draaien en fine-tunen, is er vaak nog een groot probleem. Het opslaggeheugen van de videokaart (VRAM) is vaak niet groot genoeg. De parameters van een model worden standaard opgeslagen met hoge precisie (16-bit of 32-bit kommagetallen), dit neemt al snel 15 tot 30 gigabyte aan VRAM in beslag (Dettrmers e.a., 2023).

Om dit op te lossen is Quantized LoRA (QLoRA) ontwikkeld (Dettmers e.a., 2023). Quantisatie is een techniek waarbij de precisie van de originele parameters wordt afgerond naar 4-bit of 8-bit getallen (Dettmers e.a., 2023). Dit is vergelijkbaar met het verkleinen van de resolutie van een afbeelding: de kwaliteit neemt een beetje af, maar het bestand wordt enorm veel kleiner. QLoRA combineert de twee technieken:

- Het basismodel wordt via quantisatie extreem verkleind zodat het in het geheugen van een betaalbare videokaart past (Dettmers e.a., 2023).
- Vervolgens wordt het model verder getraind via LoRA (Dettmers e.a., 2023).

2.9.4. Waarom er voor QLoRA gekozen wordt

In dit onderzoek zal er gebruik gemaakt worden van QLoRA om lokale modellen van Turtle SRL te trainen. De grootste redenen hiervoor zijn:

- **Minimale hardware:** Dankzij de combinatie van weinig trainbare parameters en eventuele quantisatie, kan het fine-tunen gebeuren met gewone consumenten-hardware (Dettmers e.a., 2023). Hierdoor heb je geen afhankelijkheid nodig van dure cloud-servers en is er geen nood om dure GPU's aan te kopen.
- **Geen verlies van algemene kennis:** Omdat het basismodel niet aangepast wordt, is er geen kans op catastrophic forgetting. Het model blijft begrijpen wat taal is, maar leert via de kleine LoRA-toevoeging hoe het ongestructureerde tekst moet omzetten in de gewenste JSON-structuur.
- **Snelheid:** In tegenstelling tot de Adapter-methode, vertraagt LoRA het genereren van de antwoorden niet. Dit is essentieel om de verwerkingstijd van 10 tot 20 seconden te verbeteren.

2.10. Evaluatiemethodes

Om objectief te kunnen vaststellen of een lokaal, gefinetuned model een volwaardig en betrouwbaar alternatief is voor de huidige cloud-aanpak, is een gestandaardiseerde evaluatiemethodiek nodig. Gewoon kijken naar hoe het model presteert in de praktijk is niet voldoende, er is nood aan reproduceerbare metrieke die toelaten de drie benaderingen eerlijk met elkaar en met de huidige cloudoplossing te vergelijken. Er zijn verschillende soorten metrieke die gebruikt kunnen worden om de prestaties van documentextractie-systemen te beoordelen (Saout e.a., 2024). Ten eerste heb je nauwkeurigheidsmetrieke die meten hoe correct de geëxtraheerde gegevens zijn. Daarnaast zijn er operationele metrieke die de prestaties van het systeem in termen van snelheid en geheugengebruik beoordelen (Saout e.a., 2024).

2.10.1. Nauwkeurigheidsmetrieken

Het resultaat van elke benadering is een JSON-object met velden zoals het factuurnummer, de leveranciersnaam, het totaalbedrag en het btw-bedrag. Om te meten hoe correct deze velden zijn, worden er meerdere metrieken gebruikt (Huang e.a., 2021).

Exact Match (EM) kijkt of een geëxtraheerd veld volledig overeenkomt met de verwachte waarde. Als het model bijvoorbeeld "€ 120,50" schrijft maar de correcte waarde "€120,50" is, telt dit als fout. Dit is een duidelijke, eenvoudige maatstaf, maar wel heel streng. (Rajpurkar e.a., 2016)

Field-level F1-score is een andere manier om te meten hoe goed een model presteert bij het uitlezen van documenten (Gantala e.a., 2025). In plaats van alleen te checken of een antwoord 100% klopt (zoals bij Exact Match), splitst de F1-score de prestaties op in twee helderen onderdelen: precisie en recall (Powers, 2011).

Om te begrijpen hoe dit werkt, kunnen we kijken naar een voorbeeld. Als we een model vragen om het aantal ordernummers uit een factuur te halen en er zijn ordernummers aanwezig maar het model geeft er maar 8 terug. Van deze 8 zijn er 6 correct en 2 ervan zijn telefoonnummers, de precisie kijkt dus naar hoeveel van de gegeven antwoorden ook echt juist zijn (Powers, 2011). In dit geval had het model dus 6 van de 8 antwoorden correct, wat een score van 75% oplevert. De recall kijkt naar hoeveel correcte antwoorden het model heeft gevonden in vergelijking met het totaal aantal correcte antwoorden (Powers, 2011). In dit geval was dit 6 van de 10, wat een recall van 60% oplevert.

In de praktijk is het vaak moeilijk om voor beiden goed te scoren. Als je een model heel streng maakt zodat het geen foute antwoorden mag geven, wordt het te voorzichtig en mist het twijfelachtige velden. Laat je een model juist heel vrij, dan zal het veel antwoorden geven maar ook veel fouten maken (Powers, 2011).

De F1-score brengt deze twee aspecten in balans (Gantala e.a., 2025). Het is een speciaal soort gemiddelde, het krijgt pas een hoge score als het zowel juiste antwoorden geeft (hoge precisie) als ook veel van de correcte antwoorden vindt (hoge recall). Het wordt berekend door de precisie en recall met elkaar te vermenigvuldigen, deze uitkomst te verdubbelen en vervolgens het totaal te delen door de som van beide waarden (Powers, 2011).

Character Error Rate (CER) meet hoe ver een geëxtraheerde waarde afwijkt van de correcte waarde, teken per teken. Concreet telt het hoeveel tekens toegevoegd, verwijderd of vervangen moeten worden om de fout te corrigeren, gedeeld door het totaal aantal tekens in de correcte waarde (K e.a., 2024). Een CER van 0 betekent een foutloze extractie. Dit is nuttig voor numerieke velden zoals bedragen en factuurnummers, waarbij één fout cijfer al een volledig verkeerde waarde oplevert (K e.a., 2024).

Naast de inhoudelijke correctheid wordt ook de JSON Schema Compliance gemeten. Dit geeft aan welk percentage van de outputs geldige JSON is die ook voldoet

aan het verwachte schema van Turtle SRL. Een model dat soms ongeldige JSON genereert is onbruikbaar in productie, ongeacht hoe nauwkeurig de andere outputs zijn (R. Han e.a., [2023](#)).

2.10.2. Operationele prestatie metrieken

Naast de nauwkeurigheid zijn ook de prestaties van het systeem een belangrijk evaluatiecriterium. De trage verwerkingstijd is immers één van de hoofdredenen om de cloud-aanpak te vervangen.

De **verwerkingstijd** is de gemiddelde tijd in seconden die nodig is om één factuur volledig te verwerken (Chitty-Venkata e.a., [2024](#)). Hiermee kan elke benadering direct worden vergeleken met de huidige benchmark van 10 tot 20 seconden.

Het **VRAM-gebruik** geeft aan hoeveel geheugen de videokaart maximaal gebruikt tijdens de verwerking (Song e.a., [2024](#)). Dit bepaalt of een benadering überhaupt haalbaar is op de bestaande hardware van Turtle SRL, zonder dat er dure uitbreidingen nodig zijn (Rajendran e.a., [2025](#)).

De **doorvoersnelheid** geeft aan hoeveel facturen het systeem per minuut kan verwerken (Chitty-Venkata e.a., [2024](#)). Dit is een maatstaf voor de schaalbaarheid wanneer het volume aan documenten toeneemt.

3

Methodologie

Met dit onderzoek wordt er een antwoord gezocht op de centrale onderzoeksvraag en de deelvragen die hierbij horen. Het onderzoek zal een totaal van 12 weken in beslag nemen, beginnend vanaf 2 maart 2026 tot en met 22 mei 2026. Het documenteren van alle uitgevoerde zaken, doorheen de fasen, zal een zeer belangrijk onderdeel zijn.

3.1. Literatuurstudie

De eerste fase van het onderzoek bestaat uit een literatuurstudie die als theoretische basis dient voor de verdere verwerking van de bachelorproef. Deze fase zal gebeuren voor het onderzoek zelf begint, en zal ongeveer 2 weken in beslag nemen. Het doel hiervan is om een goed begrip te krijgen van de huidige stand van zaken rond het automatisch verwerken van PDF-bestanden, OCR-technieken en de verschillende benaderingen voor het extraheren van gestructureerde data. Deze zal volgende vragen beantwoorden:

- Wat zijn de huidige methoden voor het extraheren van gestructureerde data uit PDF-bestanden en hoe gaan ze te werk?
- Welke fine-tuning technieken bestaan er voor LLM's met een beperkt aantal parameters?
- Welke fine-tuning techniek is het meest geschikt voor dit onderzoek?
- Welke soorten PDF-bestanden bestaan er en hoe verschillen deze in termen van opmaak en structuur?

3.2. Voorbereidingsfase

Voordat de vergelijking van de technieken kan beginnen, is er een voorbereidingsfase nodig. Hier zal er gekeken worden naar de huidige werking bij het verwerken van facturen bij Turtle SRL. Er zal vooral gelet worden op de gebruikte tools, de problemen die hierbij komen kijken en de huidige verwerkingstijd.

Daarnaast zal er samen met Turtle SRL nagegaan worden welke dataset er gebruikt zal worden voor het onderzoek. Er zal gezocht worden naar een realistische dataset van facturen met verschillende layouts en opmaakstijlen. Daarna zal er bekeken worden hoe en waar we deze drie benaderingen kunnen implementeren. De mogelijkheden hiervoor zullen beperkt worden tot de cloud, en hun eigen hardware. De volgende stap is dan het bekijken welke LLM's er gebruikt zullen worden voor dit onderzoek, de verschillende mogelijkheden zullen al onderzocht zijn in de literatuurstudie, maar er moet nog beslist worden welke dan echt gebruikt zullen worden. Deze voorbereidingsfase zal ongeveer 2 weken in beslag nemen.

3.3. Implementatiefase

Tijdens de implementatiefase worden de drie benaderingen voor factuurextractie gerealiseerd in de vorm van proof-of-concepts (PoC's). Het doel van deze fase is het bouwen van drie volledig werkende systemen die elk een andere aanpak implementeren.

Allereerst zullen de drie werkomgevingen opgezet worden, inclusief de benodigde hardware en software. Vervolgens zullen alle drie de LLM's gefinetuned worden met behulp van LoRA op de gekozen dataset van facturen. Het is belangrijk dat elke benadering dezelfde dataset gebruikt om een eerlijke vergelijking te garanderen. Voor de multimodale aanpak zal er een pipeline ontwikkeld worden die PDF-bestanden eerst omzet naar afbeeldingen. Deze afbeeldingen zullen dan vervolgens aangeboden worden aan een multimodaal LLM dat zowel visuele als tekstuele input kan verwerken. Hierbij zal de nadruk liggen op het aantal resources en de verwerkingstijd die nodig is voor een vast aantal facturen.

Voor de tekstgebaseerde aanpak zal er een systeem geïmplementeerd worden dat de PDF-bestanden eerst omzet naar een gestructureerde tekstuele representatie, zoals XML. Indien het om digitale PDF's gaat, zal de tekst direct geëxtraheerd worden. Daarna zullen deze gegevens aangeboden worden aan een LLM dat getraind is op tekstuele input. Hierbij zal er ook gekeken worden naar de verwerkingstijd en het geheugengebruik, maar ook naar het verschil in nauwkeurigheid bij digitale en gescande PDF's.

Voor de hybride aanpak zal er een combinatie van beide voorgaande pipelines ontwikkeld worden. Eerst wordt er OCR en lay-outanalyse toegepast op de PDF-bestanden om de visueel complexe zones te identificeren. Deze zones zullen dan aangeboden worden aan een multimodaal model, terwijl de rest van de tekstuele

input zal verwerkt worden door een tekst-gebaseerd LLM. Hierbij zal er ook gekeken worden naar de verwerkingstijd, het geheugengebruik en de nauwkeurigheid. Deze implementatiefase zal ongeveer 6 weken in beslag nemen.

3.4. Evaluatie- en Conclusiefase

In de evaluatiefase worden de resultaten van de drie benaderingen geanalyseerd en vergeleken. Het doel van deze fase is het objectief vergelijken van de drie ontwikkelde PoC's op basis van de vooraf bepaalde criteria. De evaluatie zal in eerste instantie kwantitatief zijn, waarbij er gekeken wordt naar de nauwkeurigheid, verwerkingstijd en het geheugengebruik van elke benadering. Om de nauwkeurigheid te meten, zal er per sleutelveld (zoals factuurnummer, bedrag, ...) gekeken worden naar het percentage correct geëxtraheerde gegevens. Deze kwantitatieve gegevens zullen visueel voorgesteld worden in grafieken en tabellen om de verschillen tussen de benaderingen duidelijk te maken.

Naast de kwantitatieve analyse zal er ook een kwalitatieve evaluatie uitgevoerd worden. Hierbij zal er onderzocht worden welke soorten fouten het meest voorkomen per benadering en hoe robuust elke methode is bij het verwerken van verschillende layouts en opmaakstijlen.

Tot slot zullen deze resultaten vergeleken worden om een aanbeveling te kunnen doen over welke benadering het meest geschikt is voor het extraheren van gestructureerde data uit PDF-bestanden in de context van Turtle SRL. Deze evaluatie- en conclusiefase zal ongeveer 2 weken in beslag nemen.

4

Vorbereidingsfase

4.1. Opzet van de testomgeving

Voor de uitvoering van dit onderzoek was de keuze van de testomgeving een bewuste beslissing. Oorspronkelijk was het de bedoeling om alle experimenten uit te voeren op een Macbook Pro met een M4 Pro-chip en 24GB unified memory. Hoewel dit systeem technisch geschikt is voor het uitvoeren van de experimenten met Ollama, zou het geen representatief beeld geven van de werkelijke situatie bij Turtle SRL. Apple Silicon maakt gebruik van unified memory, wat betekent dat de CPU en GPU hetzelfde geheugen delen, wat een andere architectuur is dan de traditionele x86-systemen met een discrete GPU die in de meeste bedrijfsomgevingen voorkomen.

Om die reden werd er gekozen om de experimenten uit te voeren op het workstation dat Turtle SRL effectief zou inzetten voor de productieomgeving. Dit systeem is uitgerust met een Intel Core i7-9800X processor, 32GB werkgeheugen en een NVIDIA GeForce RTX 3070 met 8GB VRAM. Deze hardware is representatief voor wat een KMO zoals Turtle SRL realistisch gezien zou kunnen aankopen of al in gebruik heeft. De resultaten die uit dit onderzoek voortvloeien, zijn daardoor direct toepasbaar en vergelijkbaar met de werkelijke productieomgeving van het bedrijf, wat de praktische relevantie van het onderzoek vergroot.

Een belangrijk aandachtspunt bij dit systeem is de beperkte hoeveelheid VRAM van 8GB. Zoals beschreven in de stand van zaken, vereisen modellen met meerdere miljarden parameters een grote hoeveelheid geheugen. Dankzij kwantisatie kunnen de modellen worden teruggebracht tot een grootte die binnen de capaciteit past. Voor de tekstgebaseerde aanpak wordt gebruik gemaakt van Qwen3-8B, voor de multimodale aanpak is dit Qwen3-VL-8B-Instruct. Beide modellen worden lokaal gehost via Ollama, waardoor er geen internetverbinding of externe API nodig is en de data volledig binnen de bedrijfsinfrastructuur blijft.

Het besturingssysteem van de workstation is Ubuntu 24.04.04 LTS, wat een stabiele en veelgebruikte omgeving is voor lokale AI-toepassingen. De NVIDIA-drivers en CUDA-toolkit zijn geïnstalleerd zodat Ollama de GPU optimaal kan benutten voor inferentie. Alle python-scripts worden beheerd en uitgevoerd via uv, een moderne packagemanager die zorgt voor reproduceerbare omgevingen zonder conflicten tussen afhankelijkheden.

De geëxtraheerde JSON-uitvoer van beide pipelines wordt opgeslagen in een lokale MongoDB-instantie. Elk document in de databank bevat naast de geëxtraheerde velden ook metadata zoals de gebruikte pipeline, het model en de verwerkingstijd. Dit maakt het mogelijk om de resultaten van de verschillende benaderingen achteraf systematisch te vergelijken zonder dat er data verloren gaat tussen de experimenten.

4.2. Dataset en ground truth

4.2.1. Samenstelling van de dataset

Voor dit onderzoek wordt er gebruik gemaakt van twee afzonderlijke documentverzamelingen: een testdataset voor de evaluatie van de pipelines en een trainingsdataset voor het fine-tunen van het model. Samen omvatten beide verzamelingen 99 Italiaanse facturen, verdeeld over vijf categorieën: elektriciteit, brandstof, aardgas, afval en water. Alle facturen zijn afkomstig van leveranciers waarmee Turtle SRL effectief samenwerkt en zijn uit de periode 2023 tot 2025.

4.2.2. Testdataset

De testdataset bestaat uit 25 facturen die uitsluitend worden gebruikt voor de evaluatie van de drie pipelines. Elke factuur is voorzien van een bijhorend `.json`-bestand met de manueel opgestelde ground truth. Tabel 4.1 geeft een overzicht van de verdeling per categorie, inclusief het onderscheid tussen digitale en gescande PDF's.

Categorie	Aantal	Digitaal	Gescand
Elektriciteit	5	5	0
Brandstof	5	1	4
Aardgas	5	3	2
Afval	5	4	1
Water	5	5	0
Totaal	25	18	7

Tabel 4.1: Verdeling van de testdataset per categorie

Van de 25 testfacturen zijn er 18 digitaal met volledig extracteerbare tekst. De an-

dere zeven facturen zijn gescande PDF's waarvoor OCR noodzakelijk is.

4.2.3. Trainingsdataset

De trainingsdataset bestaat uit 74 facturen die volledig losstaan van de testdataset. Door beide datasets strikt gescheiden te houden, wordt vermeden dat het fine-getuned model tijdens de evaluatie beoordeeld wordt op facturen die het al gezien heeft tijdens het trainen. Dit is essentieel voor een eerlijke en realistische beoordeling van elke methode. Tabel 4.2 geeft een overzicht van de verdeling per categorie.

Categorie	Aantal	Digitaal	Gescand
Elektriciteit	15	14	1
Brandstof	15	4	11
Aardgas	14	10	4
Afval	15	10	5
Water	15	15	0
Totaal	74	53	21

Tabel 4.2: Verdeling van de trainingsdataset per categorie (gecorrigeerd)

Opvallend in de trainingsdataset is het hoge aantal gescande facturen in de brandstofcategorie: 11 van de 15 brandstoffacturen zijn daar gescand. Dit maakt de brandstofcategorie de meest uitdagende categorie voor de tekstgebaseerde pipeline, die voor gescande documenten volledig afhankelijk is van Tesseract OCR. Ook in de afval- en aardgascategorieën komen gescande en digitale facturen door elkaar voor. De watercategorie is de enige waarbij alle facturen in de trainingsdataset digitaal zijn.

4.2.4. Gecombineerd overzicht

Tabel 4.3 geeft een gecombineerd overzicht van beide datasets. In totaal bestaat de volledige documentverzameling uit 99 unieke facturen, waarvan 71 digitaal gegenereerde PDF's (72%) en 28 gescande PDF's (28%).

	Testset	Trainingset	Totaal
Totaal PDF's	25	74	99
Digitaal	18 (72%)	53 (72%)	71 (72%)
Gescand	7 (28%)	21 (28%)	28 (28%)

Tabel 4.3: Gecombineerd overzicht van test- en trainingsdataset

4.2.5. Variatie en representativiteit

De facturen in beide datasets vertonen een grote variatie in opmaak, structuur en complexiteit, wat de dataset representatief maakt voor de realiteit waarmee Turtle SRL dagelijks geconfronteerd wordt.

- **Elektriciteit:** facturen met tot twaalf maandelijkse consumptieperioden per POD-code, inclusief een uitsplitsing per tijdsvak. De facturen zijn afkomstig van verschillende leveranciers die elk een eigen factuurindeling hebben.
- **Brandstof:** facturen met meerdere tankbeurten per voertuigidentificatiecode voor diesel (GASOLIO) en benzine (EURO 95). Het hoge aandeel gescande facturen in de trainingsdataset stelt de robuustheid van de OCR-stap specifiek voor deze categorie op de proef.
- **Aardgas:** facturen van leveranciers die zowel maandelijkse of kwartaalverbruiken hebben.
- **Afval:** facturen met Europese afvalcode (CER-codes), hoeveelheden in kilogram en verwerkingsmethoden, waarbij gescande en digitale facturen door elkaar voorkomen in beide datasets.
- **Water:** de enige categorie waarbij alle facturen digitaal zijn. Elke factuur bevat meerdere historische consumptieperioden met per periode het verbruik in kubieke meter, een gemiddeld dagverbruik en de begin- en einddatum. De periode varieert per leverancier van maandelijks tot halfjaarlijks.

4.2.6. Categorie-specifieke JSON-schema's

Een belangrijk kenmerk van deze dataset is dat de pipelines niet worden gevraagd om generieke factuurvelden te extraheren zoals een factuurnummer of een totaalbedrag. In plaats daarvan wordt er per categorie een specifiek JSON-schema gehanteerd dat uitsluitend de relevante data bevat die Turtle SRL nodig heeft voor het duurzaamheidsrapport. De vijf schema's zijn als volgt opgebouwd:

Elektriciteit, aardgas en water

Voor deze drie categorieën wordt een lijst van verbruikrecords geëxtraheerd. Elk record bevat de volgende velden:

- `codice` — de POD- of meteridentificatie van het aansluitpunt
- `indirizzo` — het leveringsadres
- `consumo` — het totale verbruik, uitgedrukt in kWh of m³
- `consumo_f1`, `consumo_f2`, `consumo_f3` — uitsplitsing per tijdsvak (uitsluitend voor elektriciteit)

- `giorno_inizio / giorno_fine` — begin- en einddatum van de meetperiode
- `costo_periodo` — de kosten voor de huidige facturatieperiode

Brandstof

Voor brandstoffacturen wordt een lijst van tankbeurten geëxtraheerd. Elk record bevat:

- `codice / tipologia` — de voertuigidentificatie en het brandstoftype (bijv. GAS-OLIO of EURO 95)
- `quantita / prezzo` — de hoeveelheid in liter en het bijhorende bedrag
- `um` — de meeteenheid
- `giorno_inizio` — de datum van de tankbeurt
- `energia_fonte / carbonfootprint_fonte` — emissiedata, indien aanwezig op de factuur

Afval

Voor afvalfacturen wordt een lijst van afvalstromen geëxtraheerd, waarbij elk record volgende velden bevat:

- `codice_cer` — de Europese afvalcode (bijv. 020201)
- `quantita` — het gewicht in kilogram
- `anno` — het jaar van verwerking
- `tipo / codice_smaltimento` — het afvaltype en de toegepaste verwerkingsmethode

4.2.7. Aanmaak van de ground truth

De ground truth voor elke factuur in beide datasets werd aangemaakt via de bestaande productiepipeline van Turtle SRL. In deze pipeline wordt elke factuur verwerkt door het cloudplatform Aibilita dat zelf gebouwd is. Deze maakt gebruik van Google Gemini 2.5 Flash om op basis van een categoriespecifieke prompt de relevante velden te extraheren en terug te geven als een JSON-bestand. Dit JSON-bestand vormt het vertrekpunt voor de ground truth.

Na de automatische extractie werd elk JSON-bestand manueel gecontroleerd en waar nodig aangepast. Tijdens deze controle werd elk geëxtraheerd veld vergeleken met de originele factuur om te verifiëren of de waarden correct en volledig zijn. Het aangepaste JSON-bestand wordt vervolgens beschouwd als de correcte output voor de bijhorende factuur en wordt opgeslagen naast het bijhorende PDF-bestand in de mappenstructuur.

Het feit dat de ground truth is gebaseerd op de output van Gemini 2.5 Flash, het cloudmodel dat dit onderzoek zou moeten vervangen, is een bewuste keuze. Enerzijds garandeert dit dat de ground truth dezelfde vorm heeft en overeenkomt met de standaarden die Turtle SRL op dit moment heeft. Anderzijds betekent dit ook dat eventuele fouten van Gemini 2.5 Flash die niet werden opgemerkt tijdens de manuele controle, ook aanwezig zullen zijn in de ground truth. Dit is een beperking die in rekening gebracht moet worden bij de interpretatie van de resultaten.

4.2.8. Evaluatiemethode

De evaluatie vergelijkt de output van de pipelines veld per veld met de ground truth via een evaluatiescript. Om afrondingsverschillen op te vangen die ontstaan bij het omzetten van tekst naar getallen, worden numerieke velden zoals verbruik en bedragen vergeleken met een tolerantie van 2%. Tekst- en datumvelden worden eerst genormaliseerd, waarbij hoofdletters worden omgezet naar kleine letters en onnodige witruimte wordt verwijderd, voordat ze met elkaar worden vergeleken. Records binnen een lijst worden vergeleken op een categoriespecifiek sleutelveld. Hierdoor zal een lijst die correct is, maar in een andere volgorde staat, toch als correct worden beschouwd.

Per document worden precision, recall en F1-score berekend voor elk veld. In batchmodus worden deze scores uitgesplitst per pipeline en per categorie, zodat duidelijk wordt voor welke types facturen elke benadering het best of minst presteert.

4.2.9. Representativiteit

De dataset dekt een realistische complexiteit: meerdere consumptieperioden per factuur, nulverbruik bij seizoensgebonden categorieën zoals gas in de zomer, ontbrekende tijdsvakken voor historische perioden en soms zeer lange tabellen met twaalf maandelijkse rijen per code.

4.3. Selectie van de modellen

Op basis van de criteria en kandidaat-modellen die besproken werden in secties 2.7 en 2.8 van de stand van zaken, werden de volgende modellen geselecteerd voor dit onderzoek. Deze selectie legt de concrete keuzes vast en motiveert waarom bepaalde kandidaten verkozen werden boven anderen.

4.3.1. Tekstmodel: Qwen3-8B

Voor de tekstgebaseerde aanpak wordt er gebruik gemaakt van Qwen3-8B, een model van Alibaba. De doorslag tegenover de andere kandidaten, zoals Llama3 8B en Mistral 7B, werd gegeven door de superieure prestatie van Qwen3 op het gebied van gestructureerde data-extractie in een meertalige context. Llama 3.1 beschikt, net zoals Qwen3, over native ondersteuning voor het Italiaans, wat een belangrijke

vereiste is voor deze toepassing. Mistral is daarentegen primair geoptimaliseerd voor Engelstalige taken, waardoor deze kandidaat minder goed aan de harde taaleis van de Italiaanse dataset voldeed.

Een tweede bepalende factor was de capaciteit van het volgen van instructies. Qwen3 volgt strikte prompt-instructies consistent op en genereert betrouwbaar geldige JSON-output zonder onnodige extra tekst toe te voegen, wat essentieel is voor een stabiele pipeline.

4.3.2. Visionmodel: Qwen3-VL:8B

Voor de multimodale aanpak werd gekozen voor Qwen3-VL:8B, dit is de multimodale uitbreiding van dezelfde Qwen3-modelreeks. De doorslag tegenover LLaVa en InternVL2-8B gaf de dynamische resolutie van Qwen3-VL, waarbij de invoerresolutie automatisch aangepast wordt aan de complexiteit van het document. Dit is bijzonder relevant voor facturen met dichte tabelstructuren en kleine tekst, waarbij een vaste lage resolutie snel leidt tot onleesbare invoer voor het model.

Daarnaast scoort Qwen3-VL beter op documentbegrip dan LLaVa, dat ontworpen is voor algemene visuele taken en minder sterk presteert op gestructureerde bedrijfsdocumenten zoals facturen. InternVL2 is een sterke kandidaat, maar dit model is minder goed getraind op Italiaans.

4.3.3. Motivatie voor de gemeenschappelijke modelfamilie

Een bewuste ontwerpkeuze in dit onderzoek is dat beide modellen afkomstig zijn uit dezelfde modelfamilie. Dit vergemakkelijkt de vergelijking tussen de twee pipelines: eventuele prestatieverschillen zijn dan eerder een gevolg van het verschil in aanpak dan aan het verschil van het model zelf. Beide modellen zullen ook gebruikt worden via Ollama in hun gekwantiseerde variant en passen binnen het VRAM budget van de RTX 3070.

5

Uitvoeringsfase

5.1. Fine-tuning van de modellen

5.1.1. Overzicht en motivatie

Zoals beschreven in de stand van zaken kan een lokaal model met een beperkt aantal parameters zonder finetuning snel afwijken van het verwachte resultaat, zelfs met een uitgebreide prompt. Dit probleem, ook wel prompt instability genoemd, is het toevoegen van extra tekst buiten de JSON-structuur, het weglaten van velden of het invullen van ontbrekende waarden op basis van aannames. Fine-tuning lost dit op door de interne kennis van het model aan te passen zodat het standaard de juiste JSON-structuur genereert voor de specifieke categorieën van Turtle SRL. In dit onderzoek worden beide modellen gefinetuned: `qwen3:8b` en `qwen3-vl:8b`. Elk model wordt getraind op een gecombineerde dataset die alle vijf de categorieën bevat zodat het model leert om alle schema's te herkennen en correct te extraheren. De categorie wordt via de prompt meegegeven zodat het model altijd weet welk schema van toepassing is. Na de fine-tuning wordt elk model geëxporteerd naar het GGUF-formaat en geïntegreerd in Ollama, waarna de drie pipelines de gefinetunede modellen gebruiken in plaats van de basismodellen. Omdat de hybride pipeline gebruik maakt van dezelfde twee modellen, is na het finetunen van beide modellen ook de hybride pipeline automatisch gefinetuned.

De fine-tuning wordt uitgevoerd met QLoRA via het Unsloth framework. Zoals beschreven in de stand van zaken combineert QLoRA quantisatie van het basismodel met LoRA adapters, waardoor fine-tuning mogelijk is op consumenten hardware met een beperkt VRAM. Unsloth is specifiek geoptimaliseerd voor efficiënte fine-tuning op dit soort hardware.

5.1.2. Voorbereiding van de trainingsset

Structuur van de trainingsdata

Fine-tuning vereist input en output paren waarbij de input de documentinhoud is en de output de correcte JSON. Voor het tekstuele model bestaat de input uit de geëxtraheerde tekst van de factuur, voor het visionmodel uit afbeeldingen van de pagina's. De output is in beide gevallen het JSON bestand met de ground truth. Per factuur wordt een trainingsvoorbeeld aangemaakt dat bestaat uit drie onderdelen: de systeemprompt, de gebruikersprompt met het categoriespecifieke schema en de documentinhoud, en de verwachte JSON output op basis van het ground truth bestand. De categorie wordt dynamisch ingevuld in de gebruikersprompt, identiek aan de manier waarop de pipelines te werk gaan. Op die manier leert het model tijdens de fine-tuning hetzelfde patroon als tijdens de effectieve verwerking. Voor het tekstmodel ziet een trainingsvoorbeeld er als volgt uit:

- **Systeem:** de vaste systeemprompt die het model instrueert om uitsluitend geldige JSON terug te geven, bedragen te normaliseren naar decimale getallen en datums naar ISO 8601 formaat om te zetten.
- **Gebruiker:** het categoriespecifieke JSON schema gevolgd door de geëxtraheerde tekst van de factuur.
- **Assistent:** het correcte JSON object uit het ground truth bestand.

Voor het visionmodel is de structuur identiek, maar de gebruikersboodschap bevat naast de prompt ook de pagina-afbeeldingen als base64-strings, identiek aan de verwerking zoals in de multimodale pipeline.

Opbouw van de gecombineerde dataset

De trainingsdata voor beide modellen wordt opgebouwd vanuit de volledige trainingsdataset in `documents_training/`. Een python-script doorloopt alle vijf de categorieën en genereert voor elke factuur een trainingsvoorbeeld. De voorbeelden van alle categorieën worden gecombineerd in één dataset en willekeurig door elkaar gemengd. Dit vermijdt dat het model tijdens de training eerst alle voorbeelden van één categorie ziet en daarna pas die van een andere categorie, wat kan leiden tot een onevenwichtige training.

Conversie naar het Alpaca-formaat

Unsloth verwacht trainingsdata in het Alpaca-formaat, waarbij elk trainingsvoorbeeld volgende velden bevat:

- **instruction:** de systeemprompt
- **input:** het categoriespecifieke JSON schema gevolgd door de documentinhoud
- **output:** het correcte JSON object uit de ground truth

De gegenereerde dataset wordt geüpload naar HuggingFace Datasets zodat Unsloth ze tijdens de training kan laden.

5.1.3. Fine-tuning van het tekstmodel

Configuratie

Het basismodel wordt geladen via Unsloth in een 4 bit gekwantiseerde vorm, waardoor er voldoende ruimte overblijft voor de LoRA adapters en de geactiveerde lagen tijdens de training. De volgende QLoRA parameters worden gebruikt:

Parameter	Waarde	Toelichting
r (rank)	16	Dimensie van de LoRA-matrices
lora_alpha	32	Schaalfactor voor de LoRA-updates
lora_dropout	0.05	Regularisatie om overfitting te beperken
target_modules	q, k, v, o, gate, up, down	Doellagen voor de LoRA-adapters
load_in_4bit	True	4-bit kwantisatie via QLoRA

Tabel 5.1: QLoRA-parameters voor het tekstmodel

Trainingsparameters

De training wordt uitgevoerd via de SFTTrainer van HuggingFace, die Unsloth intern gebruikt voor supervised fine-tuning. De volgende trainingsparameters worden gebruikt:

Parameter	Waarde	Toelichting
per_device_train_batch_size	2	Aantal voorbeelden per batch
gradient_accumulation_steps	4	Effectieve batchgrootte = 8
learning_rate	2e-4	Standaard leertempo voor LoRA
num_train_epochs	3	Aantal volledige doorlopen van de dataset
warmup_steps	10	Geleidelijke opbouw van het leertempo
max_seq_length	4096	Maximale tokenlengte per voorbeeld
optimizer	adamw_8bit	8-bit AdamW voor geheugenefficiëntie
lr_scheduler	cosine	Cosinus-afbouw van het leertempo

Tabel 5.2: Trainingsparameters voor het tekstmodel

Na de training wordt de LoRA adapter samengevoegd met het basismodel via `merge_and_unload()`, zodat er een volledig gefinetuned model overblijft.

Export naar GGUF

Na het samenvoegen wordt het model geëxporteerd naar het GGUF formaat zodat het via Ollama gehost kan worden. Unsloth biedt hiervoor ingebouwde ondersteuning via `save_pretrained_gguf()`. Het model wordt geëxporteerd in 4 bit kwantisatie, dezelfde als het originele basismodel. Het geëxporteerde GGUF-bestand wordt daarna geïmporteerd in Ollama via een modelfile en beschikbaar gesteld onder de modelnaam `qwen3-8b-finetuned`.

5.1.4. Fine-tuning van het visionmodel

Configuratie

Het basismodel wordt op dezelfde manier geladen via Unsloth in 4 bit kwantisatie. De QLoRA-parameters zijn grotendeels dezelfde als die van het tekstmodel, maar er is één belangrijk verschil: naast de taalmodellagen worden ook de lagen van de visuele encoder opgenomen als doellagen voor de LoRA adapters. Dit zorgt ervoor dat het model niet alleen leert welke JSON-structuur verwacht wordt, maar ook leert om de specifieke visuele kenmerken van de factuurlay-outs beter te herkennen.

Vorbereiding van de visuele trainingsdata

Voor het visionmodel worden de trainingsvoorbeelden aangemaakt met afbeeldingen als invoer. Per factuur worden de pagina's omgezet naar afbeeldingen op 150 DPI en gecodeerd als base64-strings. Deze worden opgenomen in de trainingsvoorbeelden naast de prompt en de verwachte JSON-output. De gecombineerde dataset met alle vijf de categorieën wordt op dezelfde manier opgebouwd als bij het tekstuele model.

Trainingsparameters en export

De trainingsparameters zijn identiek aan die van het tekstmodel, met uitzondering van `max_seq_length`, dat voor het visionmodel wordt verhoogd naar 8192 tokens. Na de trainings wordt het model op dezelfde manier samengevoegd en geëxporteerd naar GGUF via Unsloth, en geïmporteerd in Ollama onder de modelnaam `qwen3-vl-8b-finetuned`.

5.1.5. Beperkingen

De fine-tuning heeft een aantal beperkingen die relevant zijn voor de interpretatie van de resultaten.

Ten eerste bestaat er het risico dat het model de categorieën door elkaar haalt wanneer de invoerprompt niet duidelijk genoeg is. Omdat alle vijf de categorieën in dezelfde fine-tuning verwerkt worden, moet het model leren om op basis van het meegegeven schema het juiste extractiepatroon toe te passen. Een foutief of ontbrekend schema in de prompt kan leiden tot een extractie die het verkeerde schema teruggeeft.

Ten tweede is de kwaliteit van de fine-tuning direct afhankelijk van de kwaliteit van de ground truth. Zoals beschreven in de sectie over de dataset zijn de ground truth bestanden gegenereerd door Gemini 2.5 Flash en daarna manueel gecontroleerd. Eventuele fouten die niet opgemerkt werden tijdens de controle, worden door de fine-tuning overgenomen als correct.

5.2. De tekstgebaseerde aanpak

5.2.1. Overzicht

De tekstgebaseerde aanpak verwerkt elke factuur in vier opeenvolgende stappen: tekstextractie uit de PDF, extractie van informatie met het taalmodel, verwerking van de output en opslag van het resultaat. Een belangrijk kenmerk van deze aanpak is dat er een onderscheid gemaakt wordt tussen digitale en gescande PDF's, zodat de meest geschikte extractiemethode automatisch geselecteerd wordt per document.

5.2.2. Stap 1: Tekstextractie

De eerste stap is het extraheren van de tekstuele inhoud uit de PDF. Hierbij wordt er eerst geprobeerd om de tekst rechtstreeks uit het bestand te lezen met pdfplumber. Deze bibliotheek is bewust van de layout van een bestand en respecteert kolommen en tabelstructuren beter dan eenvoudige extractiemethoden, wat voordelig is voor het taalmodel.

Om te bepalen of een PDF digitaal of gescand is, wordt de lengte van de geëxtraheerde tekst gecontroleerd. Wanneer pdfplumber minder dan 50 tekens teruggeeft, wordt het document beschouwd als een gescande PDF waarbij de OCR-gebaseerde aanpak nodig is.

Bij de OCR-gebaseerde aanpak wordt elke pagina van de PDF eerst omgezet naar een afbeelding op 300 DPI via de bibliotheek pdf2image. Vervolgens worden deze afbeeldingen aangeboden aan tesseract OCR. Zoals beschreven in de stand van zaken is Tesseract een open-source OCR-engine die een hoge nauwkeurigheid behaalt bij standaard gedrukte teksten. Omdat de facturen Italiaans zijn maar ook Engelse termen kunnen bevatten, worden de taalpakketten Engels en Italiaans geconfigureerd.

In beide gevallen wordt de tekst van alle pagina's samengevoegd tot één tekstblok dat als input gebruikt wordt voor de volgende stap.

5.2.3. Stap 2: Extractie van informatie met het taalmodel

De geëxtraheerde tekst wordt samen met een categoriespecifieke prompt aangeboden aan Qwen3 via Ollama. De promptstructuur bestaat uit twee delen: een vaste systeemprompt en een dynamische gebruikersprompt. De systeemprompt bevat de algemene regels en instructies voor het model:

- Het model mag uitsluitend geldige JSON teruggeven, zonder uitleg, markdown of codefragmenten.
- Ontbrekende velden worden ingevuld als null in plaats van wegelaten of ingevuld op basis van aannames.
- Bedragen worden genormaliseerd naar een decimaal getal.
- Datums worden genormaliseerd naar het ISO-8601 formaat.
- Wanneer meerdere records aanwezig zijn, worden ze allemaal opgenomen als items in een array.

De gebruikersprompt wordt dynamisch samengesteld op basis van de categorie van de factuur. Het JSON schema wordt ingevuld in een sjabloon, gevolgd door de geëxtraheerde tekst van stap 1. Op die manier krijgt het model de gewenste structuur en ook de inhoud in één aanroep.

het model wordt geconfigureerd met de volgende parameters:

- **Temperature:** 0.1, een lage waarde waardoor de output consistent blijft, wat essentieel is voor gestructureerde extractie.
- **Contextvenster:** 8192 tokens, dit is voldoende voor de meeste facturen, maar kan een beperking zijn bij zeer lange documenten met veel historische perioden.
- **JSON-modus:** Dit is ingeschakeld via `format="json"` in de Ollama aanroep. Samen met de JSON-instructies in de prompt biedt dit een dubbele garantie op correcte output.
- **Thinking:** Dit is uitgeschakeld via `think=False`, de ingebouwde redenering van Qwen3 zou voor een extra vertraging zorgen.

5.2.4. Stap 3: Verwerking van de output

De ruwe output van het model wordt eerst gecontroleerd voordat er verder mee gewerkt wordt. Eventuele markdown omhulselen worden verwijderd en enkel het tekstblok tussen de eerste en laatste accolade wordt gebruikt. Vervolgens wordt er geprobeerd om de tekst te parsen als geldige JSON.

Wanneer de parsing mislukt, wordt de fout gelogd met de eerste 500 tekens van de ruwe output als diagnostische informatie. Er is geen optie om het opnieuw uit te voeren, als het een keer mislukt zal het bestand overgeslaan worden.

5.2.5. Stap 4: Opslag

Het resultaat wordt opgeslagen in een lokale MongoDB instantie. Elk document in de databank bevat naast de geëxtraheerde data ook de metadata die nodig zijn

voor de latere evaluatie: de bestandsnaam van de originele factuur, de categorie, het gebruikte model, de naam van de pipeline, de totale verwerkingstijd en de ruwe geëxtraheerde tekst. De ruwe geëxtraheerde tekst wordt mee opgeslagen zodat eventuele extractiefouten achteraf kunnen worden teruggekoppeld aan de kwaliteit van de tekstextractie.

Wanneer de JSON parsing mislukt, wordt het document toch opgeslagen in MongoDB, maar hierbij zal opgeslagen worden dat het niet succesvol was en de geëxtraheerde tekst zal leeg zijn. Dit maakt het mogelijk om de mislukte gevallen ook te analyseren tijdens de evaluatiefase zonder dat er data verloren gaat tussen de experimenten.

5.3. De multimodale aanpak

5.3.1. Overzicht

De multimodale aanpak verschilt van de tekstgebaseerde aanpak: er wordt geen OCR uitgevoerd en er wordt geen tussenliggende tekst geproduceerd. In plaats daarvan worden de PDF-pagina's omgezet naar afbeeldingen die rechtstreeks als visuele invoer aan het vision model worden aangeboden. Het model leest de afbeeldingen en genereert direct gestructureerde JSON in één verwerking. De pipeline bestaat uit drie stappen: de omzetting van de PDF naar afbeeldingen, de aanbidding aan het vision model en de verwerking van de output.

5.3.2. Stap 1: PDF naar afbeeldingen

De PDF-pagina's worden omgezet naar afbeeldingen via de bibliotheek pdf2image, die intern gebruik maakt van poppler. Elke pagina wordt gerenderd op 150 DPI, wat een bewuste keuze is in tegenstelling tot de 300 DPI die voor de tekstuele aanpak gebruikt wordt. Vision models werken effectief op lagere resoluties omdat ze afbeeldingen als geheel interpreteren in plaats van aparte tekens te herkennen. Bovendien beperkt een lagere resolutie het geheugengebruik.

De gegenereerde afbeeldingen worden als een lijst van PIL Image objecten in het werkgeheugen bijgehouden. Ze worden niet naar een schijf geschreven, wat onnodige I/O-bewerkingen vermijdt.

Een belangrijk onderscheid met de tekstgebaseerde aanpak is dat de multimodale aanpak geen verschil maakt tussen digitale en gescande PDF's. Bij gescande documenten ontbreekt de tekstlaag volledig, maar ook bij digitale PDF's kiest deze pipeline er bewust voor om het om te zetten naar een afbeelding. Hierdoor zullen de lay-out, tabellen en opmaak zichtbaar zijn en kunnen deze door het vision model geïnterpreteerd worden zonder tussenkomst van een aparte tekstparser.

5.3.3. Stap 2: Aanbieding aan het vision model

Dit is de kernstap van de pipeline en bestaat uit vier substappen: schaling, codering, promptopbouw en de modelaanroep.

Schaling

Elke afbeelding wordt gecontroleerd op breedte. Wanneer de breedte groter is dan 1024 pixels, wordt de afbeelding geschaald zodat de breedte exact 1024 pixels is. De hoogte wordt ook aangepast zodat de beeldverhouding hetzelfde blijft. Een pagina van 1024 op 1754 pixels wordt na deze schaling 1024 op 1447 pixels.

JPEG-codering en base64

De geschaalde afbeelding wordt via een in-memory buffer opgeslagen als JPEG met een kwaliteitsinstelling van 75. Vervolgens wordt de binaire JPEG-data gecodeerd naar een base64-string. Ollama verwacht afbeeldingen als base64-strings in het images veld van het bericht. Bij PDF's met meerdere pagina's worden alle pagina's gecodeerd en als lijst meegegeven, zodat het model alle pagina's tegelijk ontvangt in één keer. De afbeeldingen worden uitsluitend in het werkgeheugen verwerkt en worden nooit naar de schijf geschreven.

Promptopbouw

De gebruikersprompt wordt dynamisch samengesteld op basis van de categorie van de factuur. Het categoriespecifieke JSON-schema wordt ingevuld in een sjabloon, aangevuld met de instructie om uitsluitend geldige JSON terug te geven. De prompt begint met de /no_think-instructie, een Qwen3-specifieke aanwijzing die de interne redenering van het model uitschakelt. Zonder deze instructie genereert het model eerst een uitgebreide interne redenering voordat het een antwoord produceert. Voor data-extractietaken is deze redenering overbodig en zou het de verwerkingstijd langer maken.

Modelaanroep

Het model Qwen3 wordt aangeroepen via Ollama met de volgende configuratieparameters:

- **Temperature:** 0.1, deze lage waarde zorgt voor een bijna voorspelbare output. Het model kiest hierbij steeds de meest waarschijnlijke woorden, wat noodzakelijk is voor een nauwkeurige extractie.
- **Context window:** 8192 tokens, bij grote bestanden kan dit limiet bereikt worden, waardoor de verwerking afgebroken kan worden.
- **JSON-modus:** Deze functie is ingeschakeld via de instelling `format="json"`. Samen met de instructies in de prompt zorgt dit voor een dubbele controle op een correct geformateerde output.

- **Thinking:** Deze optie is uitgeschakeld met `think=False`. Hiermee wordt voorkomen dat het model zijn eigen redenering uitschrijft in het resultaat, wat de output schoon houdt.

Het model verwerkt alle pagina-afbeeldingen tegelijk en genereert de JSON-output in één keer.

5.3.4. Stap 3: Verwerking van de output en opslag

De ruwe output van het model wordt op dezelfde manier verwerkt als bij de tekstgebaseerde pipeline. Eventuele markdown-omhulsels worden verwijderd, het JSON-object wordt uitgeknipt tussen de eerste en laatste accolade, en vervolgens wordt er geprobeerd om de string te parsen als geldige JSON. Wanneer de parsing mislukt, wordt de fout gelogd met de eerste 500 tekens van de ruwe output, er is opnieuw geen retry-logica.

Ook dit resultaat wordt opgeslagen in een lokale MongoDB-instantie. Elk document bevat naast de geëxtraheerde data ook metadata zoals de bestandsnaam, de categorie, het gebruikte model, de naam van de pipeline, de totale verwerkingstijd, de ruwe modeloutput en een success veld dat aangeeft of de extractie geslaagd is.

5.4. De hybride pipeline

5.4.1. Overzicht

De hybride aanpak combineert de sterke punten van de tekstgebaseerde en de multimodale aanpak. De tekstgebaseerde pipeline is snel en nauwkeurig voor digitale tekst, maar verliest tabelstructuur en visuele lay-out bij de omzetting naar tekst. De multimodale aanpak behoudt de volledige visuele lay-out, maar is trager en minder goed voor gewone tekst. De hybride aanpak lost dit op door eerst de inhoud van elke pagina te analyseren en daarna automatisch te beslissen welk model het meest geschikt is voor welk onderdeel. Hierdoor zal de tekst naar het tekstuele model gaan en visuele structuren naar het vision model. Het tekstuele model stelt op basis van beide outputs de definitieve JSON samen.

De pipeline bestaat uit zeven stappen en heeft duidelijke verschillen die afhankelijk zijn van het type PDF. Bij een digitale PDF worden tekst en tabellen rechtstreeks uitgelezen via pdfplumber, terwijl ingebedde afbeeldingen naar het vision model gestuurd worden. Bij een gescande PDF wordt Tesseract ingezet om tekstblokken te herkennen, terwijl zones zonder herkenbare tekst als visuele zones naar het vision model gestuurd worden. Beide paden komen samen in stap 5, waarna het tekstuele model de definitieve JSON genereert.

De pipeline maakt gebruik van dezelfde twee modellen die gebruikt worden door de tekstuele en multimodale aanpak. De modellen worden nooit tegelijk in het geheugen geladen, wat essentieel is om binnen het VRAM budget van 8GB te blijven.

5.4.2. Stap 1: Lay-outanalyse

De eerste stap is de beslissingsstap van de pipeline. Pdfplumber opent de PDF en analyseert de objectstructuur van elke pagina zonder dat er afbeeldingen aangeemaakt worden. Op basis van de hoeveelheid extraheerbare tekst wordt elke pagina geclassificeerd als digitaal of gescand.

Digitale pagina's

Wanneer een pagina voldoende extraheerbare tekst bevat, worden drie soorten inhoud geïdentificeerd:

- **Tabellen:** pdfplumber doorzoekt de PDF-documenten op lijnen, kolommen en cellen. Per gevonden tabel worden de celwaarden uitgelezen en omgezet naar leesbare tekst waarbij cellen gescheiden worden door een |. Deze tekst wordt opgeslagen voor het tekstuele model. Het vision model wordt niet aangeroepen voor digitale tabellen, omdat pdfplumber de celwaarden direct kan uitlezen en gestructureerd kan teruggeven.
- **Tekst buiten tabellen:** Om dubbele data te voorkomen, worden de coördinaten van de gevonden tabellen gebruikt als filter. Alleen tekstelementen die buiten deze zones vallen, worden geëxtraherd en samengevoegd tot de tekst.
- **Afbeeldingen:** Afbeeldingsobjecten die groter zijn dan 30 op 30 pixels worden geregistreerd als visuele zones die in stap 4 naar het vision model gestuurd worden. Kleine objecten worden niet meegenomen omdat hier geen nuttige data in zal staan.

Gescande pagina's

Wanneer de totale hoeveelheid digitale tekst minder dan 50 tekens bedraagt, bevat de pagina geen echte tekst en wordt deze gemarkeerd als gescand. Tabledetectie, tekstextractie en afbeeldingsextractie worden overgeslagen, dit wordt in stap 3 gedaan door Tesseract.

5.4.3. Stap 2: Selectieve PDF naar afbeelding conversie

Na de lay-outanalyse is bekend welke pagina's een afbeelding nodig hebben voor de verder verwerking:

- Gescande pagina's hebben een afbeelding nodig voor zowel Tesseract in stap 3 als het vision model in stap 4.
- Digitale pagina's met afbeeldingen hebben een afbeelding nodig voor het vision model in stap 4.
- Digitale pagina's met enkel tekst en tabellen hebben geen afbeelding nodig.

Per benodigde pagina wordt `conver_from_path()` aangeroepen met `first_page` en `last_page` ingesteld op de exacte pagina, zodat er niet onnodig gerenderd wordt. De pagina's worden gerenderd op 300 DPI, dit is hoger dan de multimodale pipeline omdat de hybride pipeline de zones nauwkeurig moet bijsnijden en een hogere resolutie hierbij meer detail biedt bij het uitknippen van de tabelregio's en afbeeldingen. Bij een volledig digitale PDF zonder afbeeldingen wordt deze stap volledig overgeslagen.

5.4.4. Stap 3: Tesseract blokanalyse voor gescande pagina's

Voor elke gescande pagina wordt Tesseract aangeroepen via `image_to_data()` met gedetailleerde uitvoer per herkend element. Tesseract werkt op vijf niveaus, waarbij niveau twee overeenkomt met een samenhangend visuele blok en niveau vijf met een individueel woord.

Per blok worden de bounding box en de herkende woorden geregistreerd. Na het doorlopen van alle elementen wordt elk blok geclassificeerd:

- **Blokken met herkende woorden** worden samengevoegd tot OCR-tekst voor het tekstuele model. Dit zijn zones zoals adressen, factuurnummers en beschrijvingen.
- **Blokken zonder herkende woorden** maar met een minimale afmeting van 50 op 20 pixels worden beschouwd als visuele zones. Tesseract kan hier geen tekst herkennen, wat een afbeelding of grafiek zal zijn. De pixelcoördinaten worden omgezet naar een formaat waarmee pdfplumber kan werken zodat deze in stap 4 verwerkt kunnen worden als de visuele zones van een digitale pagina.

5.4.5. Stap 4: Visuele zones naar het vision model

In deze stap worden alle visuele zones verwerkt, zowel degene van een digitale als een gescande pagina. Per pagina worden alle zones gecombineerd in één vision-aanroep via vier stappen.

Bijsnijden

De puntcoördinaten van elke zone worden omgezet naar pixels via de schaalfactor $DPI/72$ en het overeenkomende fragment wordt uit de afbeelding geknipt.

Combineren

Alle uitgeknipte zones van dezelfde pagina worden verticaal gestapeld tot een gecombineerde afbeelding. Tussen elke zone wordt een grijze scheidingsbalk van 8 pixels toegevoegd om de zones visueel van elkaar te onderscheiden.

Schalen en coderen

De gecombineerde afbeelding wordt geschaald naar maximaal 1280 pixels breed. Vervolgens wordt de afbeelding als JPEG met kwaliteit 85 in het werkgeheugen opgeslagen en gecodeerd naar een base64-string.

Modelaanroep

Het vision model Qwen3-vl:8b ontvangt de gecombineerde afbeelding via een Ollama aanroep per pagina. In tegenstelling tot de multimodale pipeline wordt het vision model hier niet gevraagd om volledige JSON te genereren. In plaats daarvan wordt gevraagd om de zichtbare gegevens terug te geven als `label: waarde` per regel. Dit is een bewuste keuze: de visuele zones zijn slechts uitgeknipte fragmenten van de pagina en bevatten onvoldoende context om een volledig JSON-object te genereren dat voldoet aan het categoriespecifieke schema. Door enkel te vragen wat het model ziet, zonder het te binden aan een schema, wordt de kans op hallucinaties verminderd. Het taalmodel in stap 6 is verantwoordelijk voor de definitieve JSON generatie op basis van alle beschikbare informatie.

De modelparameters zijn: `temperature=0.1`, `num_ctx=4096` en `think=False`.

Het combineren van alle zones van één pagina in één aanroep reduceert ook het aantal aanroepen: in plaats van één aanroep per zone is er maximaal één aanroep per pagina met zones.

5.4.6. Stap 5: Tekstcontext samenstellen

Na stap 4 bevat `pagina_data` alle beschikbare informatie: rechtstreeks uitgelezen tekst, opgemaakte tabeltekst en de `label: waarde`-output van het vision model. Deze informatie wordt samengesteld tot één tekstblok dat als input dient voor het taalmodel in stap 6.

Per pagina wordt een gelabeld blok aangemaakt. Voor digitale pagina's bevat dit blok de tekst die gegenereerd is van pdfplumber buiten tabellen, gevolgd door de opgemaakte tabeltekst. Voor gescande pagina's bevat het blok de Tesseract OCR-tekst uit de tekstblokken. De vision output uit stap 4 wordt achteraan toegevoegd als een apart gelabeld blok, zodat het model duidelijk kan onderscheiden welke informatie afkomstig is van direct tekstextractie en welke van het vision model.

5.4.7. Stap 6: Definitieve JSON-generatie

De volledige context uit stap 5 wordt aangeboden aan `qwen3:8b` via Ollama. Het model ontvangt zowel de rechtstreeks uitgelezen tekst als de tabellen en genereert op basis daarvan in één keer de definitieve JSON die voldoet aan het categoriespecifieke schema.

De systeemprompt is identiek aan die van de tekstgebaseerde pipeline: het model mag uitsluitend geldige JSON teruggeven, ontbrekende velden worden ingevuld als `null`, bedragen worden genormaliseerd naar decimale getallen en datums naar

ISO formaat. De gebruikersprompt bevat het categoriespecifieke JSON schema gevolgd door de volledige context.

De configuratieparameters zijn: `temperature=0.1`, `num_ctx=8192`, `format="json"` en `think=False`.

5.4.8. Stap 7: Verwerking van de output en opslag

De verwerking van de ruwe modeloutput is identiek aan die van de andere twee pipelines: markdown omhulsels worden verwijderd, het JSON object wordt uitgeknipt tussen de eerste { en de laatste }, en vervolgens wordt er geprobeerd om de string te parsen als geldige JSON. Bij een mislukte parsing wordt de fout gelogd en wordt het resultaat opgeslagen als gefaald.

Het resultaat wordt opgeslagen in een lokale MongoDB-instantie. Naast de geëxtraheerde data en de standaard metadata en de volledige tekstcontext zoals aangeboden aan het model, bevat het document ook twee extra velden die specifiek zijn voor de hybride pipeline. Het aantal gevonden tabellen en visuele zones, en het aantal gedetecteerde afbeeldingen wordt ook opgeslagen. Deze extra metadata maken het mogelijk om tijdens de evaluatie na te gaan of de complexiteit van de factuur een impact heeft op de extractiekwaliteit.

5.4.9. Beperkingen

De hybride pipeline heeft een aantal specifieke beperkingen die relevant zijn voor de interpretatie van de resultaten.

Ten eerste is de pipeline complexer dan de andere twee benaderingen. Met zeven stappen en twee modelaanroepen per factuur zijn er meer potentiële foutpunten. Een fout in de lay-outanalyse, bijvoorbeeld een tabel die niet gedetecteerd wordt, kan leiden tot een onvolledige context voor het model in stap 6, zonder dat dit als fout gemarkeerd wordt.

Ten tweede worden de twee modellen sequentieel aangeroepen, wat betekent dat de totale verwerkingstijd per factuur structureel hoger is dan bij de andere twee pipelines. Het vision model wordt aangeroepen in stap 4 en het tekstuele model in stap 6, maar nooit tegelijk.

Ten derde is de kwaliteit van de lay-outanalyse afhankelijk van de structuur van de PDF. Bij digitale PDF's die gegenereerd zijn zonder tabelstructuren, bijvoorbeeld via eenvoudige tekstpositionering, kunnen tabellen gemist worden en de inhoud behandeld worden als gewone tekst. In dat geval verliest de hybride pipeline haar voornaamste voordeel ten opzichte van de tekstgebaseerde aanpak.

6

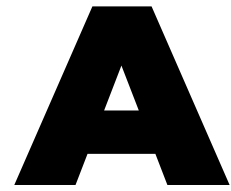
Conclusie

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Quisque enim. Proin velit neque, tristique eu, eleifend eget, vestibulum nec, lacus. Vivamus odio. Duis odio urna, vehicula in, elementum aliquam, aliquet laoreet, tellus. Sed velit. Sed vel mi ac elit aliquet interdum. Etiam sapien neque, convallis et, aliquet vel, auctor non, arcu. Aliquam suscipit aliquam lectus. Proin tincidunt magna sed wisi. Integer blandit lacus ut lorem. Sed luctus justo sed enim.

Morbi malesuada hendrerit dui. Nunc mauris leo, dapibus sit amet, vestibulum et, commodo id, est. Pellentesque purus. Pellentesque tristique, nunc ac pulvinar adipiscing, justo eros consequat lectus, sit amet posuere lectus neque vel augue. Cras consectetur libero ac eros. Ut eget massa. Fusce sit amet enim eleifend sem dictum auctor. In eget risus luctus wisi convallis pulvinar. Vivamus sapien risus, tempor in, viverra in, aliquet pellentesque, eros. Aliquam euismod libero a sem. Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

In bijna alle bedrijven zijn PDF-documenten, zoals facturen, nog steeds een cruciale bron van administratieve informatie (Smallpdf, 2024). Deze documenten bevatten vaak essentiële gegevens zoals bedragen, factuurnummers, btw-tarieven en klantgegevens, die belangrijk zijn voor de boekhouding en rapportage. Ondanks de digitalisering van veel bedrijfsprocessen, blijft het automatisch verwerken van PDF-bestanden inefficiënt en foutgevoelig. Dit komt onder andere door de verschillende opmaakstijlen en layouts die deze documenten vaak hebben.

Dit voorstel focust zich op dit probleem vanuit het perspectief van Turtle SRL, een bedrijf dat gespecialiseerd is in het optimaliseren van bedrijfsprocessen. In de huidige situatie maakt Turtle SRL gebruik van een geautomatiseerde pipeline waarbij documenten via OCR-technieken zoals Tesseract worden verwerkt in combinatie met

Google Gemini 2.5 Flash. Hoewel deze aanpak werkt, zijn hier ook een aantal problemen mee. Ten eerste is er een probleem op gebied van privacy, omdat bedrijfsgevoelige data naar Google moet verstuurd worden. Daarnaast zorgen de kosten per token en de verwerkingstijd voor schaalbaarheidsproblemen. Het duurt momenteel 10 tot 20 seconden om een factuur te verwerken. De doelgroep van dit onderzoek zijn IT-professionals en data engineers die verantwoordelijk zijn voor het implementeren van geautomatiseerde oplossingen voor documentverwerking binnen hun organisaties. De centrale onderzoeksvraag van dit voorstel is: "Welke fine-tuningbenadering van

large language models met een beperkt aantal parameters (enkele miljarden) is het meest geschikt voor het extraheren van gestructureerde data uit PDF-bestanden, zoals facturen, wanneer men drie scenario's vergelijkt: een multimodale aanpak waarbij het model zowel tekstuele als visuele input verwerkt, een tekst-gebaseerde aanpak op basis van gestructureerde representaties (zoals XML), en een hybride methode die beide combineert?"

Het doel van dit voorstel is het onderzoeken, implementeren en evalueren van deze drie benaderingen op basis van een realistische dataset van facturen. Hierbij zal een aanbeveling worden gedaan voor de methode die het meest geschikt is voor de behoeften van het bedrijf.

Om hier een goed antwoord op te kunnen geven, zullen er verschillende deelvragen onderzocht worden.

Deelvragen met betrekking tot het probleemdomein:

- Hoe is de huidige werking met Google Gemini en waar liggen de specifieke limieten?
- Welke impact heeft het gebruik van Google Gemini op de privacy van het bedrijf?
- Hoe kan de variëteit in input formaten de foutgevoeligheid van de huidige werking beïnvloeden?
- Welke soorten PDF-bestanden bestaan er en hoe verschillen deze in opmaak en structuur?

Deelvragen met betrekking tot het oplossingsdomein:

- Wat is het verschil in accuratesse tussen de tekstgebaseerde, multimodale en hybride aanpak?
- Hoe efficiënt zijn de methoden in termen van verwerkingstijd en geheugen-gebruik, vergeleken met de huidige cloudoplossing?
- In welke mate is elke benadering robuust bij het verwerken van facturen met onbekende layouts en formaten?
- Wat is de optimale balans tussen modelgrootte en prestaties?
- Welke fine-tuning techniek is het meest geschikt voor LLM's met een beperkt aantal parameters in dit specifieke domein?

A.2. Literatuurstudie

In deze literatuurstudie wordt er ten eerste nagegaan wat Turtle SRL op dit moment gebruikt om gestructureerde data uit PDF-bestanden te halen, en welke

problemen hier bij komen kijken. Vervolgens worden er drie verschillende benaderingen besproken die gebruikt kunnen worden om deze problemen aan te pakken met behulp van large language models (LLM's). Hierbij zal er ook onderzocht worden welke fine-tuning technieken er bestaan voor LLM's met een beperkt aantal parameters (enkele miljarden). Daarnaast zal er bekeken worden welke soorten PDF-bestanden er bestaan en hoe deze verschillen in termen van opmaak en structuur.

Hoewel cloud-gebaseerde LLM's momenteel de meest gebruikte oplossing zijn binnen documentenverwerking, brengen ze structurele beperkingen met zich mee voor bedrijfsapplicaties. Ten eerste is er een risico in verband met privacy aangezien alle data van het bedrijf naar een externe server wordt gestuurd en de controle hiervan wegvalt. Ten tweede zorgen de verwerkingstijden van ongeveer 10 tot 20 seconden samen met het variabele kostenmodel voor schaalbaarheidsproblemen wanneer het volume toeneemt. Om deze problemen op te lossen, kan er gebruik gemaakt worden van lokale LLM's waardoor er volledige controle is over de data en de verwerkingstijden zelf kunnen worden geoptimaliseerd.

Er zijn drie benaderingen die onderzocht zullen worden in dit voorstel: een multimodale aanpak, een tekst-gebaseerde aanpak en een hybride methode die de eerste twee combineert. Bij de multimodale aanpak wordt een PDF als een afbeelding aangeboden aan een model dat visuele elementen kan verwerken. Hierbij wordt er niet gebruik gemaakt van OCR om de tekst te extraheren, het model kan alles van deze PDF zelf verwerken. (C., 2025) toont aan dat een multimodaal model zoals DeepSeek-VL2 geschikt is voor het extraheren van informatie uit documenten aangezien het zowel tekst als visuele elementen kan verwerken. Een belangrijk nadeel van deze aanpak is dat er bij het verwerken van afbeeldingen veel rekenkracht nodig is, wat kan leiden tot lange verwerkingstijden en hoge kosten (Rajendran e.a., 2025).

De tekst-gebaseerde aanpak maakt gebruik van gestructureerde representaties van de PDF, zoals XML, om de tekstuele inhoud te extraheren. Het omzetten van een PDF naar een tekstuele representatie hiervan is dus de belangrijkste stap bij deze methode, aangezien deze tekst de input zal zijn voor de LLM. We moeten hier ook een onderscheid maken tussen twee soorten PDF's: digitale PDF's, die tekstuele informatie bevatten, en gescande PDF's, die in feite afbeeldingen zijn (ABBY, 2025). Bij digitale PDF's kan de tekst direct geëxtraheerd worden, maar hierbij zal de structuur vaak verloren gaan, waardoor tabellen moeilijk te interpreteren zijn (Stephens, 2022). Bij de gescande PDF's is Optical Character Recognition (OCR) nodig om de tekst uit de afbeelding te halen (ABBY, 2025) en deze dan aan te bieden aan een LLM. In dit onderzoek zal er gebruik gemaakt worden van Tesseract OCR, dit is een open-source OCR-engine die gebaseerd is op deep learning en gebruikt wordt in zowel academische als industriële toepassingen (Smith, 2007). Volgens (Smith, 2007) behaalt Tesseract hoge herkenningssnauwkeurigheid bij standaard

gedrukte teksten en ondersteunt het meerdere talen, waaronder Engels, Nederlands en Italiaans.

De hybride aanpak combineert bovenstaande methoden door eerst OCR en layoutanalyse toe te passen en enkel de visueel complexe zones zoals afbeeldingen aan een multimodaal model te geven. Hierdoor zal de verwerkingstijd en rekenkracht verminderd worden in vergelijking met de eerste aanpak.

Naast de keuze van de benadering is het ook belangrijk om te onderzoeken hoe een LLM kan worden aangepast naar een specifiek domein. Fine-tuning verwijst naar het verder trainen van een reeds getraind model op domeinspecifieke data („Fine-tuning Models”, 2025). Hierdoor zullen de prestaties van het model verbeteren voor deze specifieke taak („Fine-tuning Models”, 2025). Voor LLM's met een beperkt aantal parameters (enkele miljarden) is dit zeer belangrijk, aangezien deze modellen vaak minder algemene kennis bevatten dan de grotere modellen.

Er zijn verschillende technieken voor fine-tuning, zoals volledige fine-tuning, waarbij alle parameters van het model worden aangepast. Deze methode heeft wel als nadeel dat het veel rekenkracht, data en tijd vereist, zelfs bij kleinere modellen. Het gevolg hiervan is dat er op zoek is gegaan naar een andere techniek: Parameter-Efficient Fine-Tuning (PEFT). Een voorbeeld hiervan is Low-Rank Adaptation (LoRA), waarbij slechts een klein aantal extra parameters wordt toegevoegd aan het model, terwijl de originele parameters bevroren blijven. Deze methode kan vergelijkbare prestaties leveren als volledige fine-tuning, maar met veel minder rekenkracht en data. Hierdoor is het mogelijk om modellen te trainen op hardware met beperkte rekenkracht, wat essentieel is voor een kostenefficiënte implementatie. Een andere techniek is Prefix Tuning, waarbij vaste trainbare vectoren vooraan aan de invoer van het model worden toegevoegd om het gedrag te sturen zonder de originele modelparameters te wijzigen. (Z. Han e.a., 2024) Voor dit onderzoek zal er gebruik gemaakt worden van LoRA, aangezien deze techniek slechts een beperkte hoeveelheid trainingsdata vereist en minder rekenkracht nodig heeft in vergelijking met volledige fine-tuning.

A.3. Methodologie

Met dit onderzoek wordt er een antwoord gezocht op de centrale onderzoeksvraag en de deelvragen die hierbij horen. Het onderzoek zal een totaal van 12 weken in beslag nemen, beginnend vanaf 2 maart 2026 tot en met 22 mei 2026. Het documenteren van alle uitgevoerde zaken, doorheen de fasen, zal een zeer belangrijk onderdeel zijn.

A.3.1. Literatuurstudie

De eerste fase van het onderzoek bestaat uit een literatuurstudie die als theoretische basis dient voor de verdere verwerking van de bachelorproef. Deze fase zal gebeuren voor het onderzoek zelf begint, en zal ongeveer 2 weken in beslag nemen.

Het doel hiervan is om een goed begrip te krijgen van de huidige stand van zaken rond het automatisch verwerken van PDF-bestanden, OCR-technieken en de verschillende benaderingen voor het extraheren van gestructureerde data. Deze zal volgende vragen beantwoorden:

- Wat zijn de huidige methoden voor het extraheren van gestructureerde data uit PDF-bestanden en hoe gaan ze te werk?
- Welke fine-tuning technieken bestaan er voor LLM's met een beperkt aantal parameters?
- Welke fine-tuning techniek is het meest geschikt voor dit onderzoek?
- Welke soorten PDF-bestanden bestaan er en hoe verschillen deze in termen van opmaak en structuur?

A.3.2. Voorbereidingsfase

Voordat de vergelijking van de technieken kan beginnen, is er een voorbereidingsfase nodig. Hier zal er gekeken worden naar de huidige werking bij het verwerken van facturen bij Turtle SRL. Er zal vooral gelet worden op de gebruikte tools, de problemen die hierbij komen kijken en de huidige verwerkingstijd.

Daarnaast zal er samen met Turtle SRL nagegaan worden welke dataset er gebruikt zal worden voor het onderzoek. Er zal gezocht worden naar een realistische dataset van facturen met verschillende layouts en opmaakstijlen. Daarna zal er bekeken worden hoe en waar we deze drie benaderingen kunnen implementeren. De mogelijkheden hiervoor zullen beperkt worden tot de cloud, en hun eigen hardware. De volgende stap is dan het bekijken welke LLM's er gebruikt zullen worden voor dit onderzoek, de verschillende mogelijkheden zullen al onderzocht zijn in de literatuurstudie, maar er moet nog beslist worden welke dan echt gebruikt zullen worden. Deze voorbereidingsfase zal ongeveer 2 weken in beslag nemen.

A.3.3. Implementatiefase

Tijdens de implementatiefase worden de drie benaderingen voor factuurextractie gerealiseerd in de vorm van proof-of-concepts (PoC's). Het doel van deze fase is het bouwen van drie volledig werkende systemen die elk een andere aanpak implementeren.

Allereerst zullen de drie werkomgevingen opgezet worden, inclusief de benodigde hardware en software. Vervolgens zullen alle drie de LLM's gefinetuned worden met behulp van LoRA op de gekozen dataset van facturen. Het is belangrijk dat elke benadering dezelfde dataset gebruikt om een eerlijke vergelijking te garanderen. Voor de multimodale aanpak zal er een pipeline ontwikkeld worden die PDF-bestanden eerst omzet naar afbeeldingen. Deze afbeeldingen zullen dan vervolgens aangeboden worden aan een multimodaal LLM dat zowel visuele als tekstuele input kan

verwerken. Hierbij zal de nadruk liggen op het aantal resources en de verwerkingstijd die nodig is voor een vast aantal facturen.

Voor de tekstgebaseerde aanpak zal er een systeem geïmplementeerd worden dat de PDF-bestanden eerst omzet naar een gestructureerde tekstuele representatie, zoals XML. Indien het om digitale PDF's gaat, zal de tekst direct geëxtraheerd worden. Daarna zullen deze gegevens aangeboden worden aan een LLM dat getraind is op tekstuele input. Hierbij zal er ook gekeken worden naar de verwerkingstijd en het geheugengebruik, maar ook naar het verschil in nauwkeurigheid bij digitale en gescande PDF's.

Voor de hybride aanpak zal er een combinatie van beide voorgaande pipelines ontwikkeld worden. Eerst wordt er OCR en lay-outanalyse toegepast op de PDF-bestanden om de visueel complexe zones te identificeren. Deze zones zullen dan aangeboden worden aan een multimodaal model, terwijl de rest van de tekstuele input zal verwerkt worden door een tekst-gebaseerd LLM. Hierbij zal er ook gekeken worden naar de verwerkingstijd, het geheugengebruik en de nauwkeurigheid. Deze implementatiefase zal ongeveer 6 weken in beslag nemen.

A.3.4. Evaluatie- en Conclusiefase

In de evaluatiefase worden de resultaten van de drie benaderingen geanalyseerd en vergeleken. Het doel van deze fase is het objectief vergelijken van de drie ontwikkelde PoC's op basis van de vooraf bepaalde criteria. De evaluatie zal in eerste instantie kwantitatief zijn, waarbij er gekeken wordt naar de nauwkeurigheid, verwerkingstijd en het geheugengebruik van elke benadering. Om de nauwkeurigheid te meten, zal er per sleutelveld (zoals factuurnummer, bedrag, ...) gekeken worden naar het percentage correct geëxtraheerde gegevens. Deze kwantitatieve gegevens zullen visueel voorgesteld worden in grafieken en tabellen om de verschillen tussen de benaderingen duidelijk te maken.

Naast de kwantitatieve analyse zal er ook een kwalitatieve evaluatie uitgevoerd worden. Hierbij zal er onderzocht worden welke soorten fouten het meest voorkomen per benadering en hoe robuust elke methode is bij het verwerken van verschillende layouts en opmaakstijlen.

Tot slot zullen deze resultaten vergeleken worden om een aanbeveling te kunnen doen over welke benadering het meest geschikt is voor het extraheren van gestructureerde data uit PDF-bestanden in de context van Turtle SRL. Deze evaluatie- en conclusiefase zal ongeveer 2 weken in beslag nemen.

A.4. Verwacht resultaat, conclusie

Op basis van de literatuurstudie wordt er verwacht dat de hybride benadering de beste balans zal bieden tussen nauwkeurigheid en efficiëntie. Dit komt omdat deze methode de sterke punten van zowel de multimodale als de tekst-gebaseerde aanpak combineert. De multimodale aanpak zal waarschijnlijk de hoogste nauw-

keurigheid behalen bij het extraheren van gegevens uit visueel complexe facturen, maar zal ook de meeste rekenkracht en verwerkingstijd vereisen. De tekstgebaseerde aanpak zal daarentegen sneller zijn en minder resources vergen, maar kan moeite hebben met het correct interpreteren van visueel complexe elementen. Deze twee methoden zullen waarschijnlijk het interessantst zijn voor de meeste bedrijven. Er zal per bedrijf gekeken worden naar de beschikbare resources en de vereiste nauwkeurigheid om te bepalen welke van deze twee benaderingen het meest geschikt is.

Bibliografie

- ABBYY. (2025). *Types of PDFs: Searchable PDF, Image-Only, True PDF*. Verkregen december 8, 2025, van <https://pdf.abbyy.com/learning-center/pdf-types/>
- Available Models - ComfyUI-QwenVL. (2025). Verkregen mei 4, 2026, van <https://deepwiki.com/1038lab/ComfyUI-QwenVL/5.1-available-models>
- C., E. (2025). *Ultimate Guide - The Best Open Source LLM for Document Screening in 2025*. Verkregen december 8, 2025, van <https://www.siliconflow.com/articles/en/best-open-source-LLM-for-Document-screening>
- Chitty-Venkata, K. T., Mittal, S., Emani, M., Vishwanath, V., & Somani, A. K. (2024). LLM-Inference-Bench: Inference Benchmarking of Large Language Models on AI Accelerators. <https://arxiv.org/abs/2411.00136>
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. <https://arxiv.org/abs/2305.14314>
- Fine-tuning Models* [Bezocht op 11 december 2025]. (2025). OpenAI. <https://platform.openai.com/docs/guides/model-optimization>
- Gantala, S. Y., e.a. (2025). Invoice Information Extraction: Methods and Performance Evaluation. <https://arxiv.org/abs/2510.15727>
- Glukhov, R. (2025). *Ollama versus vLLM versus LM Studio: de beste manier om LLM's lokaal uit te voeren*. Verkregen mei 4, 2026, van <https://www.glukhov.org/nl/llm-hosting/comparisons/hosting-llms-ollama-localai-jan-lmstudio-vllm-comparison/>
- Han, R., Yang, C., Peng, T., Tiwari, P., Wan, X., Liu, L., & Wang, B. (2023). An Empirical Study on Information Extraction using Large Language Models. <https://arxiv.org/abs/2305.14450>
- Han, Z., Wang, G., Chen, J., Yang, Z., Wang, J., Cui, P., Wang, Y., Zhang, Y., Han, B., & Qiao, Y. (2024). Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. *arXiv preprint arXiv:2403.13261*. <https://arxiv.org/abs/2403.13261>
- Huang, Z., Chen, K., He, J., Bai, X., Karatzas, D., Lu, S., & Jawahar, C. (2021). ICDAR2019 Competition on Scanned Receipt OCR and Information Extraction. *arXiv preprint arXiv:2103.10213*. <https://arxiv.org/abs/2103.10213>
- Hugging Face. (2025). *Use Ollama with any GGUF Model on Hugging Face Hub*. Verkregen april 26, 2026, van <https://huggingface.co/docs/hub/en/ollama>

- IntraGPT. (2023). *Ollama & Lokale LLMs - On-Premise AI*. Verkregen april 26, 2026, van <https://intragpt.nl/ollama>
- K, T. D., e.a. (2024). Advocating Character Error Rate for Multilingual ASR Evaluation. <https://arxiv.org/abs/2410.07400>
- LM Studio. (2024). *LM Studio 0.3.0*. Verkregen mei 4, 2026, van <https://lmstudio.ai/blog/lmstudio-v0.3.0>
- Master, L. A. (2026). *Ollama RAM & VRAM for Every Model (Master Table)*. Verkregen april 26, 2026, van <https://localaimaster.com/blog/ollama-model-ram-vram-table>
- Mastery, M. L. (2025). *Using Quantized Models with Ollama for Application Development*. Verkregen maart 28, 2026, van <https://machinelearningmastery.com/using-quantized-models-with-ollama-for-application-development/>
- Ollama. (2026a). *Modelfile Reference*. Verkregen april 26, 2026, van <https://docs.ollama.com/modelfile>
- Ollama. (2026b). *Ollama API Documentation*. Verkregen april 4, 2026, van <https://docs.ollama.com/>
- Ollama. (2026c). *Thinking*. Verkregen april 26, 2026, van <https://docs.ollama.com/capabilities/thinking>
- Ollama. (2026d). *Vision - Ollama's documentation*. Verkregen april 4, 2026, van <https://docs.ollama.com/capabilities/vision>
- Ollama VRAM Requirements: Complete Guide to Local LLMs on Consumer GPUs*. (2026). Verkregen mei 4, 2026, van <https://localllm.in/blog/ollama-vram-requirements-for-local-llms>
- OpenGVLab. (2023). *OpenGVLab/InternVL2-8B*. Verkregen mei 4, 2026, van <https://huggingface.co/OpenGVLab/InternVL2-8B>
- Palomares Carrascosa, I. (2025). *5 Problems Encountered Fine-Tuning LLMs with Solutions*. Verkregen april 30, 2025, van <https://machinelearningmastery.com/5-problems-encountered-fine-tuning-llms-with-solutions/>
- Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness & Correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63. <https://arxiv.org/abs/2010.16061>
- Practical DevSecOps. (g.d.). *Catastrophic Forgetting*. Verkregen februari 28, 2026, van <https://www.practical-devsecops.com/glossary/catastrophic-forgetting/>
- Qwen. (2025). *Qwen3: Think Deeper, Act Faster*. Verkregen mei 4, 2026, van <https://qwen.ai/blog?id=qwen3>
- Rajendran, R. K., Debnath, B., Sankaradas, M., & Chakradhar, S. T. (2025). EcoDoc: A Cost-Efficient Multimodal Document Processing System for Enterprises Using LLMs. *Proceedings of the 63rd Annual Meeting of the Association for*

- Computational Linguistics: Industry Track*. <https://aclanthology.org/2025.acl-industry.109>
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2383–2392. <https://arxiv.org/abs/1606.05250>
- Saout, T., Lardeux, F., & Saubion, F. (2024). An Overview of Data Extraction from Invoices. *IEEE Access*. https://www.researchgate.net/publication/377865621_An_Overview_of_Data_Extraction_from_Invoices
- SitePoint. (2026). *Guide to Local LLMs in 2026: Privacy, Tools & Hardware*. Verkregen april 26, 2026, van <https://www.sitepoint.com/definitive-guide-local-llms-2026-privacy-tools-hardware/>
- Smallpdf. (2024). *PDF Statistics 2024: How People Use PDFs Today*. Verkregen december 10, 2025, van <https://smallpdf.com/pdf-statistics>
- Smith, R. (2007). An Overview of the Tesseract OCR Engine. *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. <https://research.google.com/pubs/archive/33418.pdf>
- Song, Y., Mi, Z., Xie, H., & Chen, H. (2024). PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. <https://arxiv.org/abs/2312.12456>
- Stephens, M. (2022, juli 8). *Why is PDF text extraction problematic?* IDR solutions. Verkregen december 7, 2025, van <https://blog.idrsolutions.com/why-is-pdf-text-extraction-problematic/>
- Stewart, I., Horawalavithana, S., Kennedy, B., Munikoti, S., & Pazdernik, K. (2024). Surprisingly Fragile: Assessing and Addressing Prompt Instability in Multimodal Foundation Models. *arXiv*. <https://arxiv.org/abs/2408.14595>
- Teki, S. (2024). *Mistral of Experts - Mixtral 7B*. Verkregen mei 4, 2026, van <https://www.sundeepteki.org/blog/Mistral-of-experts>
- vLLM. (2026). *vLLM Options and Optimization*. Verkregen mei 4, 2026, van <https://docs.vllm.ai/en/stable/configuration/optimization/>
- Xia, H., e.a. (2023). Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment. <https://arxiv.org/abs/2312.12148>
- Xu, L., e.a. (2024). Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. <https://arxiv.org/abs/2403.14608>
- Xu, Z., e.a. (2026). One Model, Many Skills: Parameter-Efficient Fine-Tuning for Multitask Code Analysis. <https://arxiv.org/abs/2603.09978>
- Yasuno, T. (2026). A Comparative Study of LLaVA-1.5-7B Quantization Levels. *arXiv*. Verkregen mei 4, 2026, van <https://arxiv.org/html/2603.26770v1>